

Computação Gráfica – Jogos Digitais

OpenGL GLSL

Aula 16

Professor: André Flores dos Santos



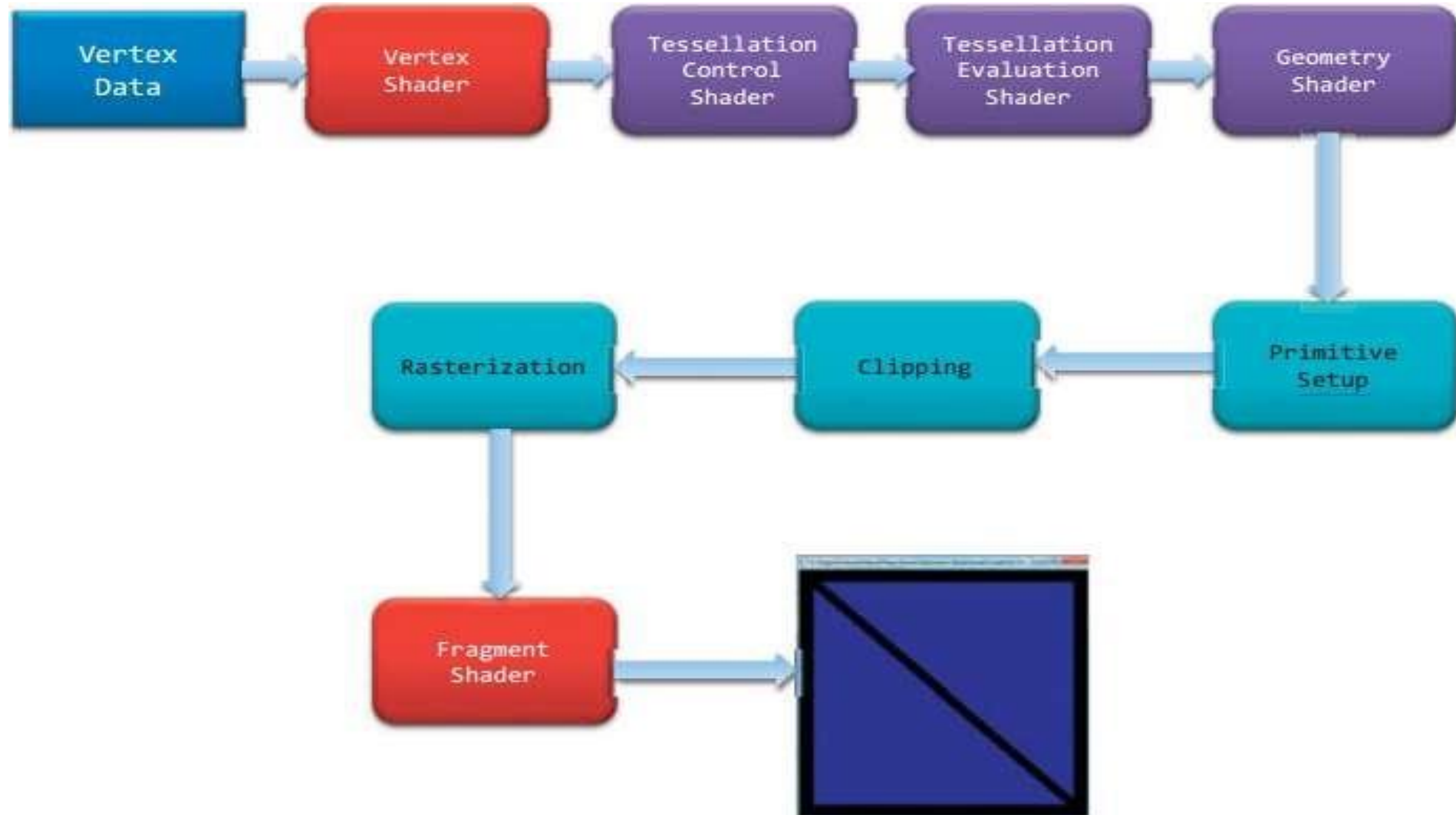
Shaders

- O **Pipeline de Renderização do OpenGL** moderno depende muito de **shaders** para o **processamento de dados**;
- Shaders passaram a ser obrigatórios a partir da versão 3.1 do OpenGL;
- Métodos anteriores de renderização foram removidos.

GLSL

- Shaders podem ser especificados na linguagem GLSL (*OpenGL Shading Language*).
- GLSL é muito similar a linguagem C/C++;
- O que estudaremos:
 - **Compilação dos shaders;**
 - **Integração;**
 - **Transferência de dados entre shaders.**

Pipeline de Renderização do OpenGL



Pipeline de Renderização do OpenGL

- Objetivo: converter os dados de objetos tridimensionais em uma imagem renderizada;
- Inicialmente, os dados geométricos da aplicação passam por uma sequência de shaders: ***vertex shader, tessellation shader e geometry shader***;
- Em seguida, na etapa ***Primitive Setup***, os dados são organizados na forma de primitivas geométricas: **pontos, linhas e triângulos**;
- No recorte (***clipping***), são retirados da renderização os objetos que estão fora do volume de visualização;
- Na rasterização, as primitivas restantes são transformadas em fragmentos de píxeis, que serão processados, finalmente, no ***fragment shader***.

Pipeline programável do OpenGL

- A versão 4.5 do pipeline gráfico do OpenGL contém 4 estágios de processamento e mais um estágio de computação, todos controlados por meio de um shader;
- Possui dois **estágios obrigatórios**:
 - **Vertex Shading**: recebe os **dados** dos vértices da **aplicação** através dos **VBOs**, processando cada vértice separadamente e definindo suas **posições finais**.
 - **Fragment Shading**: processa **fragmentos individuais** gerados pelo **rasterizador** do OpenGL. Nesse estágio, os valores referentes a **cor** e **profundidade** são computados.

Fluxo de dados entre shaders

- É importante entender como os dados fluem de sua **aplicação** para um **shader**, bem como de um **shader** para **outro**;
- Isso é feito por meio de **variáveis globais** especiais declaradas na especificação de um shader;
- Para isso, vamos analisar o código a seguir referente a um ***vertex shader***:

Exemplo

```
1. #version 400
2. in vec4 vPosicao;
3. in vec4 vCor;
4. out vec4 cor;
5. uniform mat4 MatrizVisualizacaoProjecao;
6. void main()
7. {
8.     cor = vCor;
9.     gl_Position = MatrizVisualizacaoProjecao * vPosicao;
10. }
```


Análise do exemplo

- Geralmente o shader **começa** com uma declaração da sua **versão** utilizando **#version**;
- As **variáveis globais** do shader são utilizadas pelo OpenGL para **transferir dados** entre os shaders, e além de serem de um **tipo de dado específico** (**vec3**, **vec4**, **mat3**, **mat4**, etc.), elas possuem algumas propriedades:
 - Variáveis que “**entram**” no shader são declaradas com **in**, enquanto que as variáveis que “**saem**” do shader são declaradas com **out**.
 - Esses valores são atualizados toda vez que o OpenGL executa o shader:
 - ▮ No **vertex shader**, são atualizadas para cada **vértice**;
 - ▮ No **fragment shader**, são atualizadas para cada **fragmento**;

Análise do exemplo_(cont.)

- As variáveis que o OpenGL recebe de uma aplicação são variáveis do tipo **uniform**:
 - Variáveis do tipo **uniform**, nos shaders, são **somente para leitura**.
 - A variável **MatrizVisualizacaoProjecao** é uma matriz 4x4 transferida da aplicação para o shader, representando, provavelmente, uma matriz que combina as transformações de **visualização** (câmera) e de **projeção**.
- Por fim, define-se o valor da variável **gl_Position**, que é uma variável especial de saída do **vertex shader** que define a **posição final daquele vértice**.

Criação de Shaders com GLSL

- Um shader inicia sua execução da mesma forma que um programa em C, por meio da função `main()`:

```
#version 400
void main()
{
    // Seu código vai aqui
}
```

Declaração de variáveis

- GLSL é uma linguagem tipada, ou seja, toda variável precisa ser declarada e ter um **tipo de dado associado**:

Tipo	Descrição
float	Variável de ponto flutuante de 32 bits
double	Variável de ponto flutuante de 64 bits
int	Inteiro de 32 bits com sinal
uint	Inteiro de 32 bits sem sinal
bool	Variável booleana

Inicialização de variáveis

- Variáveis podem ser inicializadas ao serem declaradas:

```
int i, numParticles = 1500;  
float force, g = -9.8;  
bool falling = true;  
double pi = 3.141592;
```

Tipos agregados

- GLSL possui alguns **tipos agregados** de dados, que são uma **combinação** de vários dados de um mesmo tipo;
- GLSL suporta **vetores** de dois, três e quatro componentes para cada um dos tipos básicos (**bool**, **int**, **uint**, **float** e **double**);
- Além disso, **matrizes** do tipo **float** e **double** também estão disponíveis, conforme mostra a tabela do próximo slide.

Tipos agregados

Table 2.3 GLSL Vector and Matrix Types

Base Type	2D vec	3D vec	4D vec	Matrix Types		
float	vec2	vec3	vec4	mat2	mat3	mat4
				mat2x2	mat2x3	mat2x4
				mat3x2	mat3x3	mat3x4
				mat4x2	mat4x3	mat4x4
double	dvec2	dvec3	dvec4	dmat2	dmat3	dmat4
				dmat2x2	dmat2x3	dmat2x4
				dmat3x2	dmat3x3	dmat3x4
				dmat4x2	dmat4x3	dmat4x4
int	ivec2	ivec3	ivec4	—		
uint	uvec2	uvec3	uvec4	—		
bool	bvec2	bvec3	bvec4	—		

Obs: os tipos de matrizes definidos com duas dimensões, tal como `mat4x3`, utilizam o primeiro valor para especificar a quantidade de colunas, e o segundo valor a quantidade de linhas.

Inicialização/atribuição

- Vetores podem ser inicializados de várias formas, por exemplo:
 - Inicialização simples, definido o valor de cada componente:

```
vec3 velocidade= vec3(0.0, 2.0, 3.0);
```
 - Convertendo um vetor de 3 elementos do tipo **float** para um vetor de 3 elementos do tipo **int**:

```
ivec3 steps = ivec3(velocidade);
```
 - Trucando vetores de uma dimensão maior para vetores de dimensões menores:

```
vec4 cor;  
vec3 RGB = vec3(cor); //agora RGB possui somente 3 elementos
```
 - Outros tipos de inicialização/conversão/truncamento:

```
vec3 branco = vec3(1.0); //branco = (1.0,1.0,1.0)  
vec4 translucido = vec4(branco, 0.5); //translucido=(1.0,1.0,1.0,0.5)
```


Inicialização/atribuição

- Matrizes também podem ser inicializadas de diversas maneiras:

- Inicialização como uma matriz diagonal:

```
mat3 m = mat3(4.0); // m =  $\begin{pmatrix} 4.0 & 0.0 & 0.0 \\ 0.0 & 4.0 & 0.0 \\ 0.0 & 0.0 & 4.0 \end{pmatrix}$ 
```

- Inicialização como uma matriz totalmente populada:

```
mat3 m = mat3(1.0, 2.0, 3.0,  
              4.0, 5.0, 6.0,  
              7.0, 8.0, 9.0)
```

Inicialização/atribuição

- Inicialização a partir da combinação de vetores:

```
▮ vec3 coluna1 = vec3(1.0, 2.0, 3.0);  
▮ vec3 coluna2 = vec3(4.0, 5.0, 6.0);  
▮ vec3 coluna3 = vec3(7.0, 8.0, 9.0);  
▮ mat3 m = mat3(coluna1, coluna2, coluna3);
```

- Ou até mesmo combinando diferentes dimensões:

```
▮ vec2 coluna1 = vec2(1.0, 2.0);  
▮ vec2 coluna2 = vec2(4.0, 5.0);  
▮ vec2 coluna3 = vec2(7.0, 8.0);  
▮ mat3 M = mat3(coluna1, 3.0,  
                 coluna2, 6.0,  
                 coluna3, 9.0);
```

Acesso e manipulação

- Os elementos individuais de vetores podem ser acessados de duas formas:

- Por meio de um componente nominal:

```
vec4 cor = vec4(0.6, 0.4, 0.3, 0.5);  
vec3 velocidade = vec3(1.0, 1.0, 0.0);  
float vermelho = cor.r;  
float v_y = velocidade.y;
```

- Ou por meio de um índice de array, como estamos acostumados:

```
float vermelho = cor[0];  
float v_y = velocidade[1];
```

Acesso nominal de vetores

- Para o acesso nominal de vetores, a GLSL possui **3 conjuntos de nomenclaturas**, sendo que todos eles tem o mesmo efeito:

Componente	Descrição
(x, y, z, w)	Componentes associados a posições
(r, g, b, a)	Componentes associados a cores
(s, t, p, q)	Componentes associados a coordenadas de texturas

- Esses 3 conjuntos são uteis para esclarecer o **tipo de operação** que está sendo realizada sobre os dados.

Acesso nominal de vetores

- Com o acesso nominal, podemos tirar bastante proveito para realizar operações do tipo:
 - `vec3 cor = vec3(0.6, 0.5, 0.2);`
 - `vec3 luminancia = cor.rrr;`
 - *`//luminancia = (0.6, 0.6, 0.6)`*
- Ou ainda:
 - `vec3 corInvertida = cor.bgr`
 - *`//corInvertida = (0.2, 0.5, 0.6)`*

Acesso nominal de vetores

- Mas precisamos ter o cuidado de NÃO fazer coisas do tipo:
 - **Misturar grupos** de nomes diferentes:
 - ▮ `vec3 cor = outraCor.rgz`
 - ▮ *//erro: z é de um grupo diferente!*
 - Tentar acessar um elemento **fora do alcance**:
 - ▮ `vec2 pos = vec2(0.45, 0.12);`
 - ▮ `float zPos = pos.z;`
 - ▮ *//erro: não existe componente z em um vetor de 2 dimensões*

Acesso a matrizes

- **Elementos de uma matriz** só podem ser acessados na sua forma de **array**. Apesar disso, a linguagem GLSL nos traz algumas facilidades:

//declaração de uma matriz diagonal

```
mat4 m = mat4(2.0);
```

// zVec recebe a segunda coluna da matriz

```
vec4 zVec = m[2];
```

// acessar m[1].y também tem o mesmo resultado

```
float yScale = m[1][1];
```

Estruturas

- Semelhantemente a linguagem C, podemos especificar **estruturas** em GLSL;
- Estruturas, ao serem definidas, automaticamente cria-se um **novo tipo de dado** e um **construtor implícito** com seus parâmetros na **ordem** em que foram **especificados**:

//definição

```
struct Particula {  
    float tempoDeVida;  
    vec3 posicao;  
    vec3 velocidade;  
};
```

//instanciamento

```
vec3 pos = vec3(5.0, 2.0, -3.0);  
vec3 vel = vec3(10.0, -3.0, 0.0);  
Particula p = Particula(10.0, pos, vel);
```

//acesso

```
float t = p.tempoDeVida;
```

//manipulação

```
p.Velocidade = vec3(50.0, -23.0, 0.0);
```


Arrays

- Arrays simples também podem ser especificados, assim como na linguagem C:

```
//um array de 3 floats  
float vetor[3];  
//idem  
float[3] vetor_b;  
/* array sem tamanho definido, que deve ser redeclarado no futuro com um tamanho */  
int indices[];  
//array declarado com valores já atribuídos  
float vetor_C[3] = float[3](2.38, 3.14, 42.0);  
//varrendo um array  
for (int i = 0; i < vetor3.length(); i++) {  
    vetor3[i] *= 2.0;  
}
```

Matrizes

- Cuidado ao utilizar o método `length()` em matrizes:

```
//declaração de uma matriz de 3 colunas e 4 linhas  
mat3x4 m;
```

```
//c recebe o número de colunas de m: 3  
int c = m.length();
```

```
//r recebe o numero de elementos na coluna 0: 4  
int r = m[0].length();
```

Modificadores

- Os tipo de dados também podem possuir modificadores que afetam seu comportamento;
- Existem quatro tipos de modificadores em GLSL, conforme mostra a tabela abaixo:

Modificador	Descrição
const	Variável de valor constante, ou seja, que serve somente para leitura
in	Especifica uma variável de entrada de um shader
out	Especifica uma variável de saída de um shader
uniform	Especifica uma variável cujo valor é passado para o shader através da aplicação, e é constante para toda a primitiva desenhada

Operadores

- Quase todos os operadores da linguagem C estão presentes na linguagem GLSL;
- O que os torna interessante é o fato de que podemos aplicá-los aos mais diferentes tipos de dados, como por exemplo, para fazer a multiplicação de vetores, matrizes, etc.

Table 2.6 GLSL Operators and Their Precedence

Precedence	Operators	Accepted types	Description
1	()	—	Grouping of operations
2	[]	arrays, matrices, vectors	Array subscripting
	f()	functions	Function calls and constructors
	. (period)	structures	Structure field or method access
	++ --	arithmetic	Post-increment and -decrement
3	++ --	arithmetic	Pre-increment and -decrement
	+ -	arithmetic	Unary explicit positive or negation
	~	integer	Unary bit-wise not
	!	bool	Unary logical not
4	* / %	arithmetic	Multiplicative operations
5	+ -	arithmetic	Additive operations
6	<< >>	integer	Bit-wise operations
7	< > <= >=	arithmetic	Relational operations
8	== !=	any	Equality operations
9	&	integer	Bit-wise and
10	^	integer	Bit-wise exclusive or
11		integer	Bit-wise inclusive or
12	&&	bool	Logical and operation
13	^^	bool	Logical exclusive-or operation
14		bool	Logical or operation
15	a ? b : c	bool ? any : any	Ternary selection operation (inline “if” operation; if (a) then (b) else (c))
16	=	any	Assignment
	+= -=	arithmetic	Arithmetic assignment
	*= /=	arithmetic	
	%= <<= >>=	integer	
	&= ^= =	integer	
17	, (comma)	any	Sequence of operations

Operações

- Multiplicação de vetor por matriz:

```
vec3 v;
```

```
mat3 m;
```

```
vec3 result = v * m;
```

- Multiplicação de matriz por matriz:

```
mat2 m, u, v;
```

```
m = u * v;
```

- O único cuidado que precisamos ter é de **respeitar a dimensionalidade da multiplicação!**

Controle de fluxo

- As estruturas de controle da linguagem GLSL são as mesmas de sempre: **if-else** e **switch-case**:

```
if (verdade) {  
    // cláusula verdadeira  
}  
else {  
    // cláusula falsa  
}
```

```
switch (valor_inteiro) {  
    case n:  
        // ...  
        break;  
    case m:  
        // ...  
        break;  
    default:  
        //...  
        break;  
}
```

Laços de repetição

- GLSL suporta os laços de repetição **for**, **while** e **do ... while**:

```
for (int i = 0; i < 10; i++) {  
    //...  
}
```

```
while (n < 10) {  
    //...  
}
```

```
do {  
    //...  
} while (n < 10);
```

Funções

- Funções permitem que você substitua códigos recorrentes com chamadas a funções;
- GLSL fornece uma série de funções para facilitar o processamento dos dados, as quais podem ser facilmente encontradas na sua documentação:
 - <https://www.opengl.org/sdk/docs/man4/index.php>
- E claro, podemos também definir as nossas funções.

Declaração de funções

- A declaração de uma função possui uma sintaxe semelhante ao C, com a diferença de que podemos utilizar qualificadores de acesso, conforme a estrutura abaixo:

```
tipoDeRetorno nomeDaFuncao(  
    [qualificadorDeAcesso1] tipo1 parametro1,  
    [qualificadorDeAcesso2] tipo2 parametro2,  
    ...)  
{  
    //corpo da função  
  
    return valorDeRetorno //a não ser que o retorno seja void  
}
```

Em GLSL os qualificadores de parâmetro são:

in: só entra na função (read-only).

out: só sai da função (write-only).

inout: entra e sai — você pode ler e depois escrever de volta para o chamador.

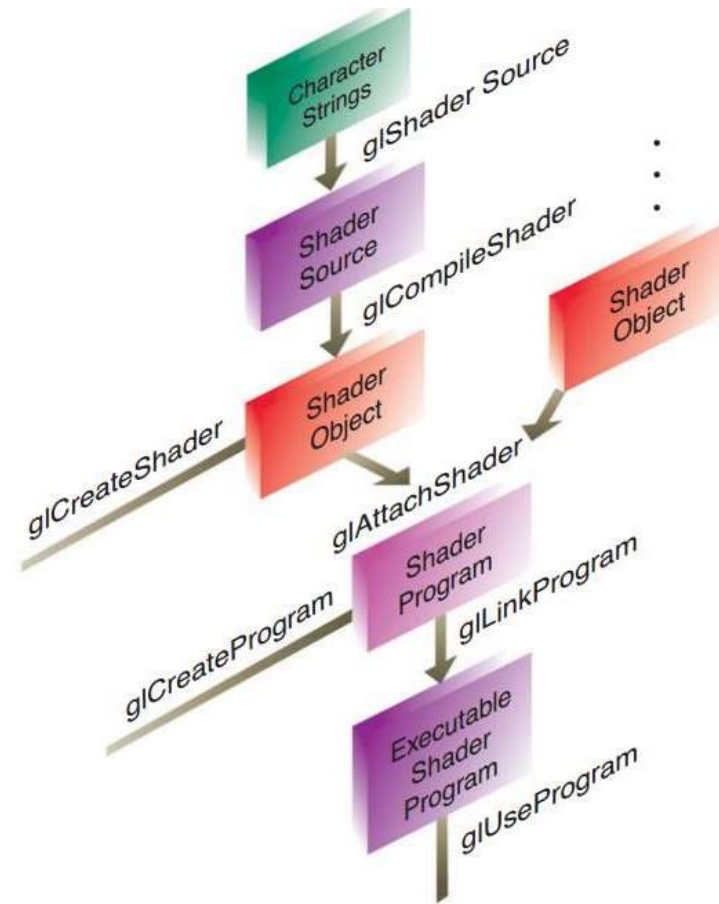
Compilando shaders

Shaders possuem um compilador semelhante a linguagem C, que analisa o código, verifica se existem erros, e então traduz o código para um **código objeto**;

- Em seguida, nós combinamos uma coleção de objetos de modo que os mesmos gerem um **programa executável**;
- A diferença é que o compilador da linguagem GLSL faz parte da API do OpenGL;
- A figura a seguir ilustra os passos necessários para a criação de ***shader objects*** e para a ligação de vários ***shader objects*** para a criação de um ***shader program***.

Compilando shaders

- Para a criação de um shader program, precisamos seguir os seguintes passos:
 - Para cada shader object:
 - ▮ Criamos um **shader object**;
 - ▮ Compilamos o código fonte do shader para um código objeto (**shader object**)
 - ▮ Verificamos se houve algum erro de compilação
 - Então, para linkar múltiplos **shader objects** em um único **shader program**, nós:
 - ▮ Criamos um **shader program**;
 - ▮ Anexamos os shader objects ao **shader program**;
 - ▮ Linkamos o **shader program**;
 - ▮ Verificamos se a linkagem ocorreu corretamente;
 - ▮ Utilizamos o **shader program** para o processamento dos vértices/fragmentos



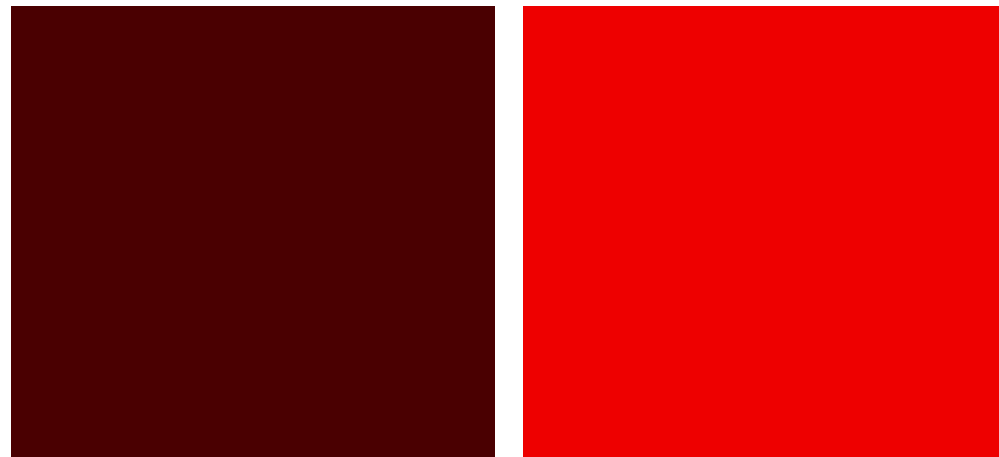
Exemplos

```
/*Adicionamos o bloco #ifdef GL_ES ... precision mediump float;
para definir a precisão de floats em ambientes WebGL/GL ES.*/
```

```
#ifdef GL_ES
precision mediump float;
#endif
```

```
// Uniform injetado pelo glsl-canvas
uniform float u_time;
```

```
void main() {
    // calcula um pulso vermelho baseado no tempo
    float pulso = abs(sin(u_time));
    gl_FragColor = vec4(pulso, 0.0, 0.0, 1.0);
}
```



Esse shader faz com que a tela (ou objeto) fique preta quando $\sin(\text{time})$ está em zero, e passe para vermelho intenso quando $\sin(\text{time})$ chega a ± 1 , pois estamos realizando esse cálculo no componente 'R' do RGB do `gl_FragColor`.

https://github.com/andreflores2009/ComputacaoGrafica_2025-01_JD/blob/main/Exercicios/Aula17/Ex1_time_r.glsl

Exemplos

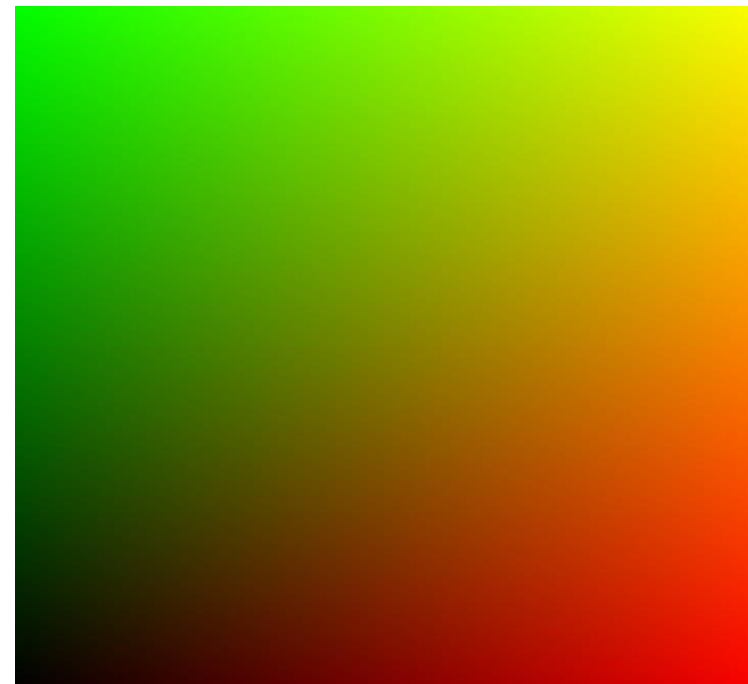
```
#ifdef GL_ES
precision mediump float;
#endif
```

```
/* a variável "resolution" vem da aplicação e contem a
resolução da viewport */
uniform vec2 u_resolution;
```

```
void main() {
```

```
/* a variável gl_FragCoord armazena as coordenadas do fragmento na tela */
vec2 st = gl_FragCoord.xy/u_resolution;
gl_FragColor = vec4(st.x,st.y,0.0,1.0);
}
```

Resultado visual: um gradiente que vai do preto no canto inferior esquerdo (0,0), passando por vermelho puro no canto inferior direito (1,0), verde puro no canto superior esquerdo (0,1) e amarelo (vermelho+verde) no canto superior direito (1,1).



https://github.com/andreflores2009/ComputacaoGrafica_2025-01_JD/blob/main/Exercicios/Aula17/Ex2.glsl

Normalização Exemplo 02

```
precision mediump float;
```

```
// Uniforme para armazenar a resolução da tela  
uniform vec2 u_resolution;
```

```
void main() {
```

```
    // gl_FragCoord contém as coordenadas do pixel na viewport/janela, enquanto  
    // u_resolution contém a resolução da tela.
```

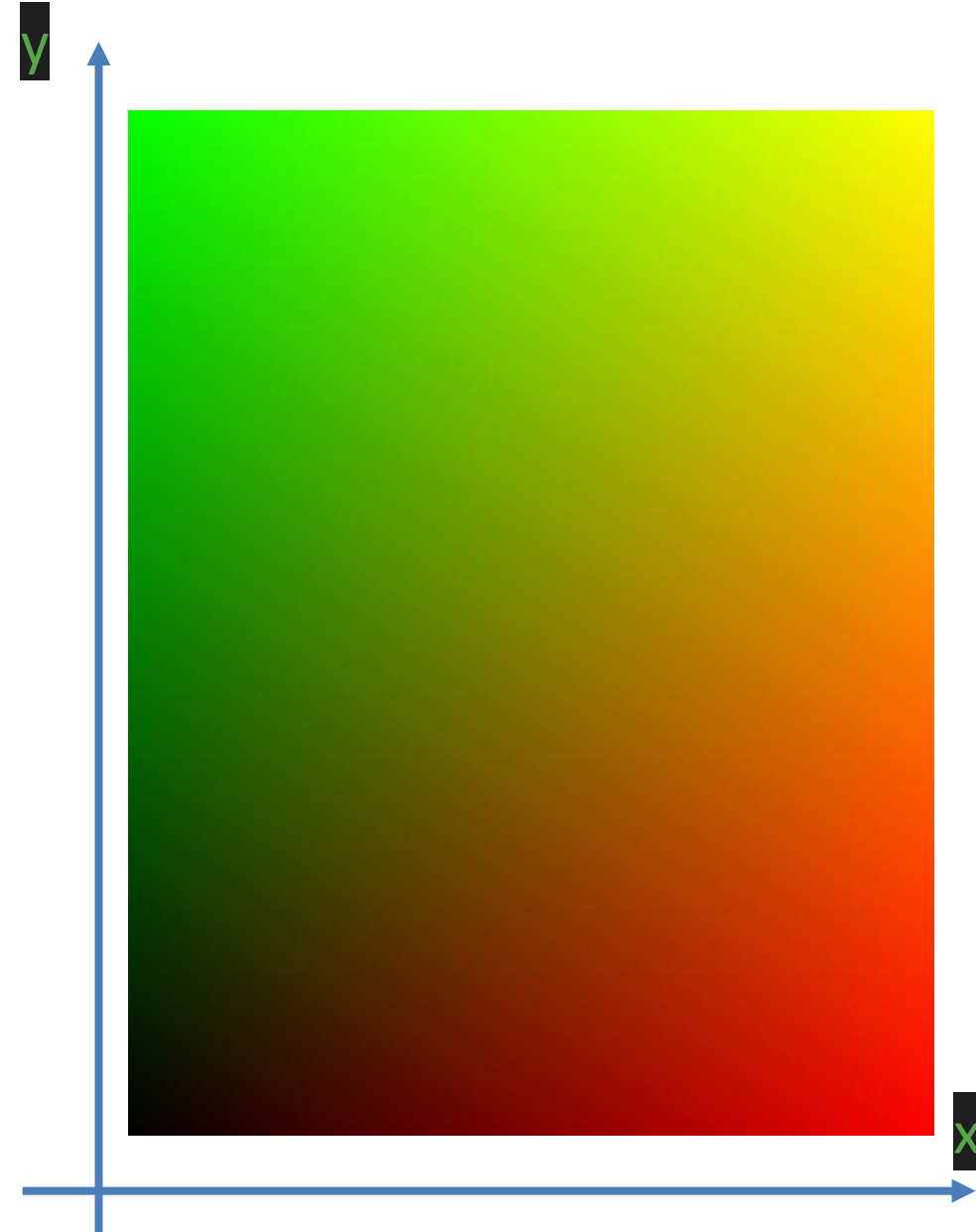
```
    // A variável st recebe a posição do pixel normalizada, de modo que sua nova  
    // faixa de valores passe a ser de 0 a 1, tanto na largura quanto na altura.
```

```
    vec2 st = gl_FragCoord.xy / u_resolution;
```

```
    // A cor do fragmento é definida com base nas coordenadas normalizadas,  
    // atribuindo a coordenada x ao canal vermelho (R), a coordenada y ao canal  
    // verde (G), e o canal azul (B) é definido como 0. O canal alfa (A) é  
    // definido como 1, o que significa que o fragmento é completamente opaco.
```

```
    gl_FragColor = vec4(st.x, st.y, 0.0, 1.0);
```

```
}
```



Shaders em OpenGL

Integração com o Pipeline de Renderização:

Os shaders em OpenGL são parte de um pipeline de renderização mais amplo que inclui configuração do contexto OpenGL, buffers de vértices, texturas e a lógica de renderização da aplicação.

Você escreve código de aplicação em uma linguagem como C++ ou Python para configurar o OpenGL, carregar dados, compilar shaders e controlar o fluxo de renderização.

Shaders em OpenGL

Estrutura do Código:

- Incluem vertex shaders, fragment shaders, e possivelmente outros tipos de shaders (geometry shaders, tessellation shaders, compute shaders) que são ligados juntos em programas de shader.
- Necessitam de variáveis uniform, attribute (ou in e out nas versões mais recentes), e funções específicas como main.

Contexto OpenGL:

- Requer um contexto OpenGL ativo para compilar e executar shaders.
- Utiliza bibliotecas como GLFW, SDL, ou outras para criar janelas e contextos gráficos.

Shaders em OpenGL

Vamos tentar aplicar no OpenGL o efeito variando do verde para o vermelho. Do Exemplo 01 que fizemos.

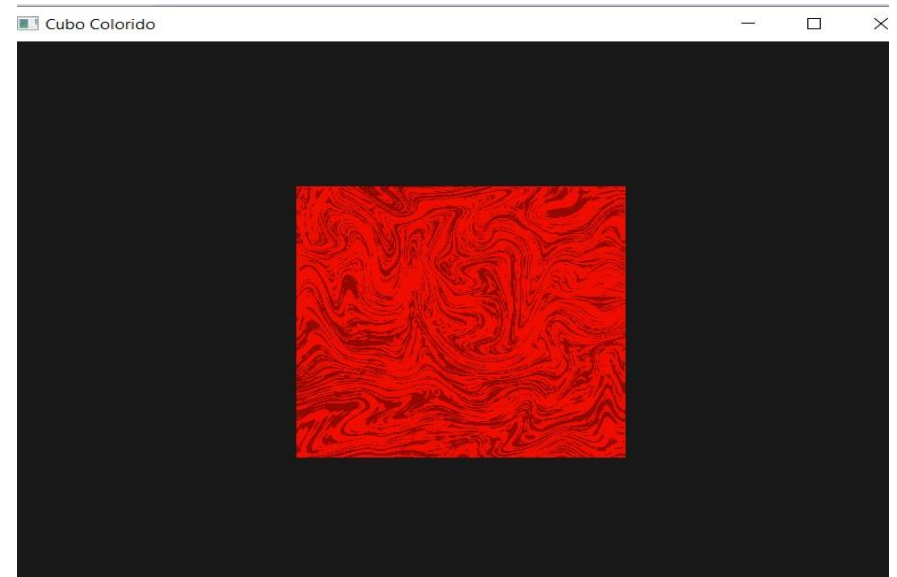
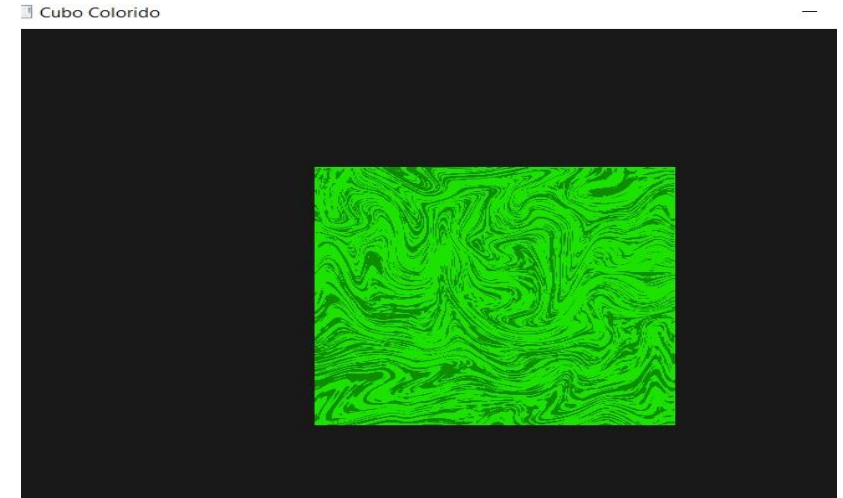
Contexto

- **Objetivo:** Aplicar o efeito de “pulso” de verde→vermelho num cubo 3D usando shaders

- **Ambientes:**

- `glsl-canvas` (WebGL/GL ES)
- **OpenGL core 4.0** (PyOpenGL + GLFW + Pyrr)

https://github.com/andreflores2009/ComputacaoGrafica_2025-01_JD/blob/main/Exercicios/Aula17/Shader_Textura_Cubo.py



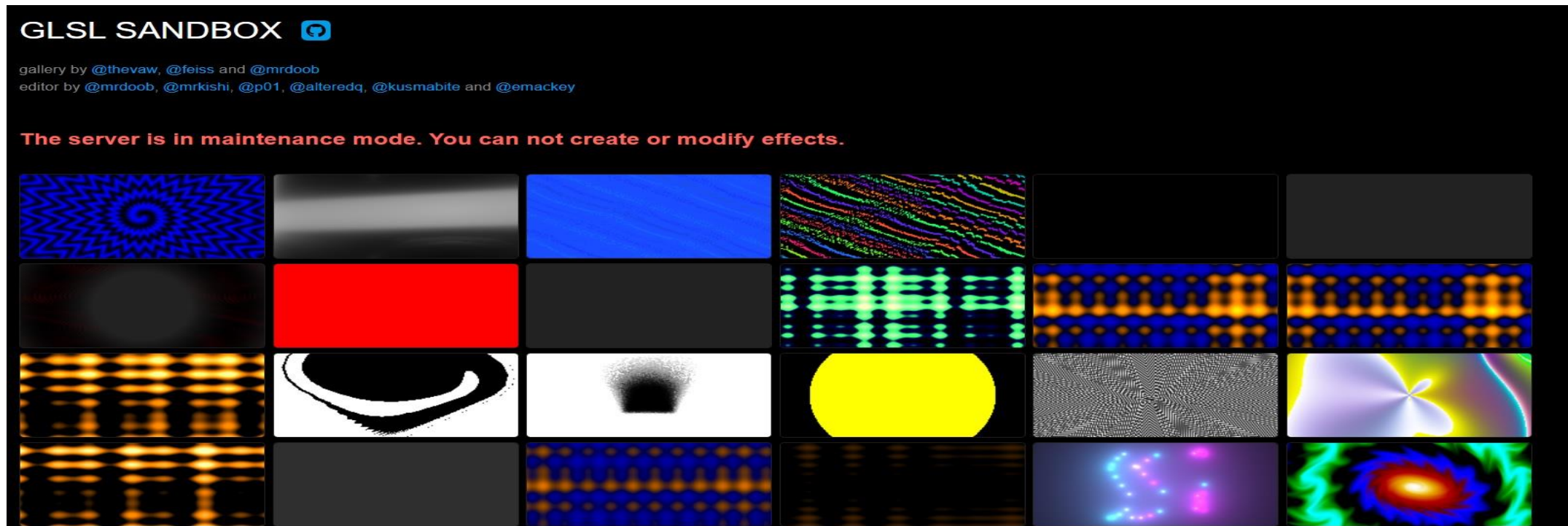
Shaders em OpenGL

Diferenças de Setup

Aspecto	glsl-canvas	OpenGL core 4.0 (PyOpenGL)
Versão GLSL	Implícita (WebGL padrão)	#version 400 core
Precision	precision mediump float;	(não obrigatório em desktop)
Saída de cor	gl_FragColor = ...;	out vec4 FragColor;
Uniforms automáticos	u_time, u_resolution	Você mesmo faz glGetUniformLocation + glUniform...
Texture unit	Implícita no plugin	glActiveTexture(GL_TEXTURE0); glUniform1i(loc, 0);

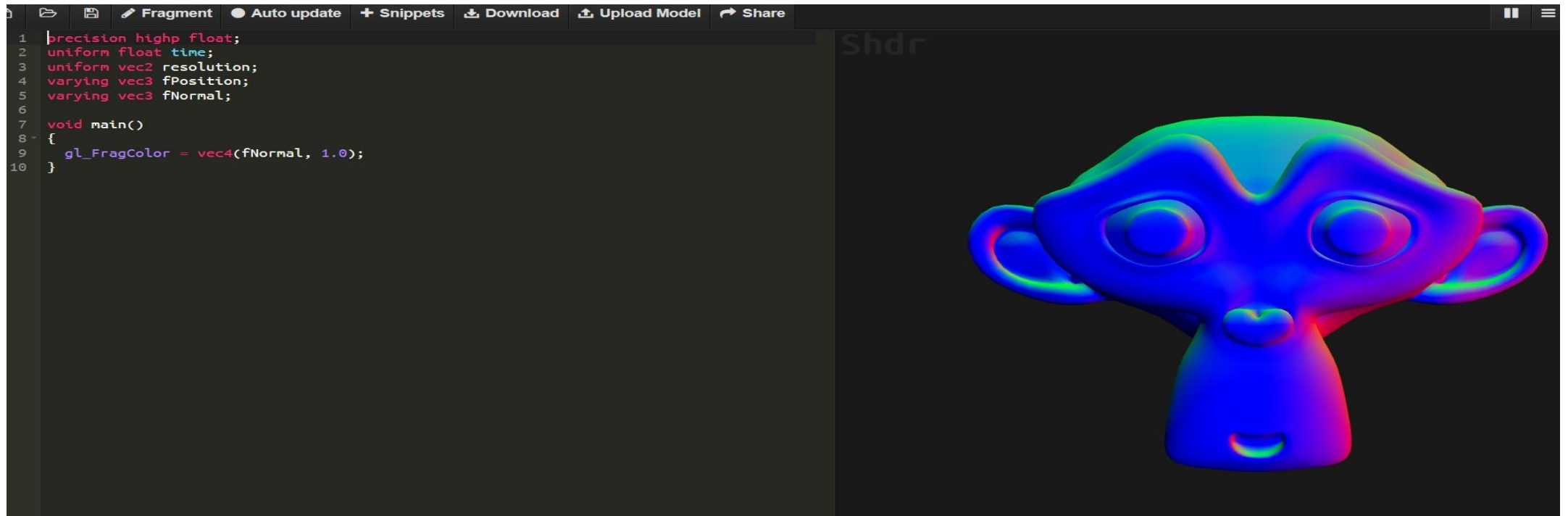
Testando Shaders

- Existem alguns editores online que permitem que você teste seus shaders diretamente do navegador, tal como:
 - <http://glslsandbox.com/>



Testando Shaders

- Existem alguns editores online que permitem que você teste seus shaders diretamente do navegador, tal como:
 - <http://shdr.bkcore.com/>



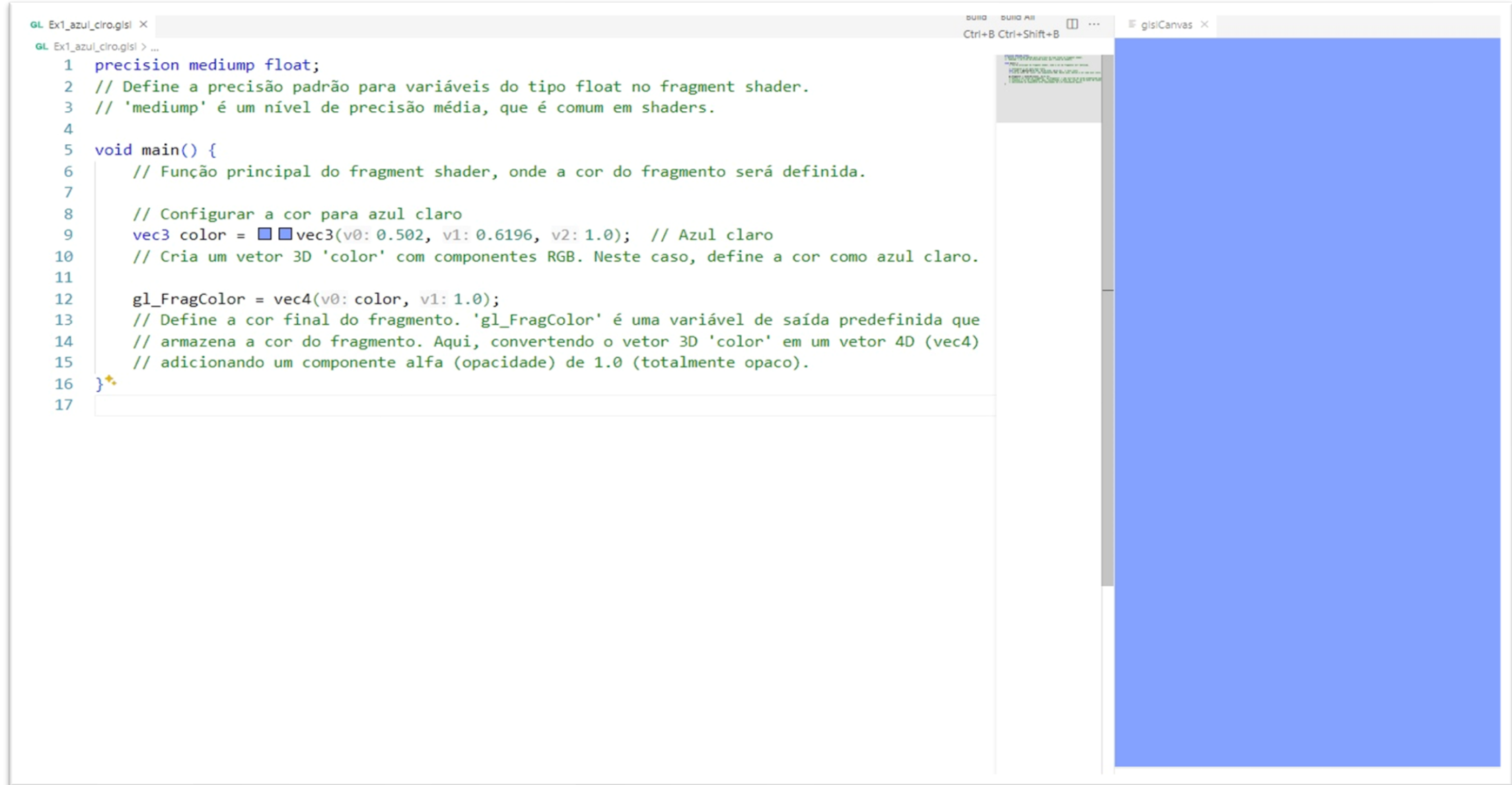
Shaders em OpenGL GLSL-Canvas (VSCode)

Execução Isolada:

- Os shaders são executados isoladamente dentro do ambiente da extensão, sem a necessidade de configurar um contexto OpenGL manualmente.
- O GLSL Canvas cria automaticamente um contexto WebGL para visualizar o resultado do shader, simplificando o desenvolvimento e a prototipagem.

Shaders em OpenGL GLSL- Canvas (VSCode)

Exemplo:



The image shows a screenshot of the Visual Studio Code editor. On the left, a file named 'GL Ex1_azul_ciro.glsl' is open, displaying the following GLSL code:

```
1 precision mediump float;
2 // Define a precisão padrão para variáveis do tipo float no fragment shader.
3 // 'mediump' é um nível de precisão média, que é comum em shaders.
4
5 void main() {
6     // Função principal do fragment shader, onde a cor do fragmento será definida.
7
8     // Configurar a cor para azul claro
9     vec3 color = vec3(v0: 0.502, v1: 0.6196, v2: 1.0); // Azul claro
10    // Cria um vetor 3D 'color' com componentes RGB. Neste caso, define a cor como azul claro.
11
12    gl_FragColor = vec4(v0: color, v1: 1.0);
13    // Define a cor final do fragmento. 'gl_FragColor' é uma variável de saída predefinida que
14    // armazena a cor do fragmento. Aqui, convertendo o vetor 3D 'color' em um vetor 4D (vec4)
15    // adicionando um componente alfa (opacidade) de 1.0 (totalmente opaco).
16 }
17
```

On the right, a canvas window titled 'glslCanvas' displays the result of the shader: a solid, light blue rectangle.

Shaders em OpenGL GLSL- Canvas (VSCode)

Agora vamos testar alguns exemplos no nosso ambiente VS CODE+ glsl Canvas

Computação Gráfica – Jogos Digitais

OpenGL GLSL - Shaders

Aula 16 - Parte II

Professor: André Flores dos Santos

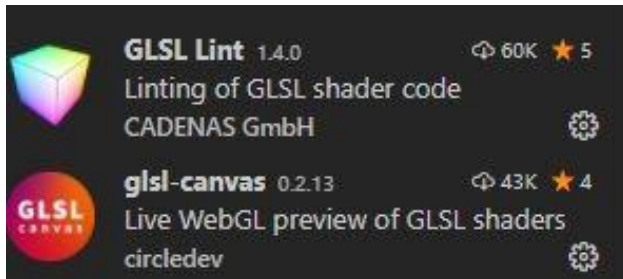


GLSL

- GLSL (OpenGL Shading Language) é uma linguagem de programação padrão para shaders, sendo esta específica para a plataforma OpenGL.
- GLSL é muito similar a linguagem C/C++;

Ferramentas

- Instale as seguintes extensões no Visual Studio Code para trabalharmos diretamente com GLSL:



- Para exibir o resultado da execução do shader em tempo real, pressione F1 e digite *Show gls/Canvas*

Exemplo 03 – Exibir a Cor azul

```
precision mediump float;
// Define a precisão padrão para variáveis do tipo float no fragment shader.
// 'mediump' é um nível de precisão média, que é comum em shaders.

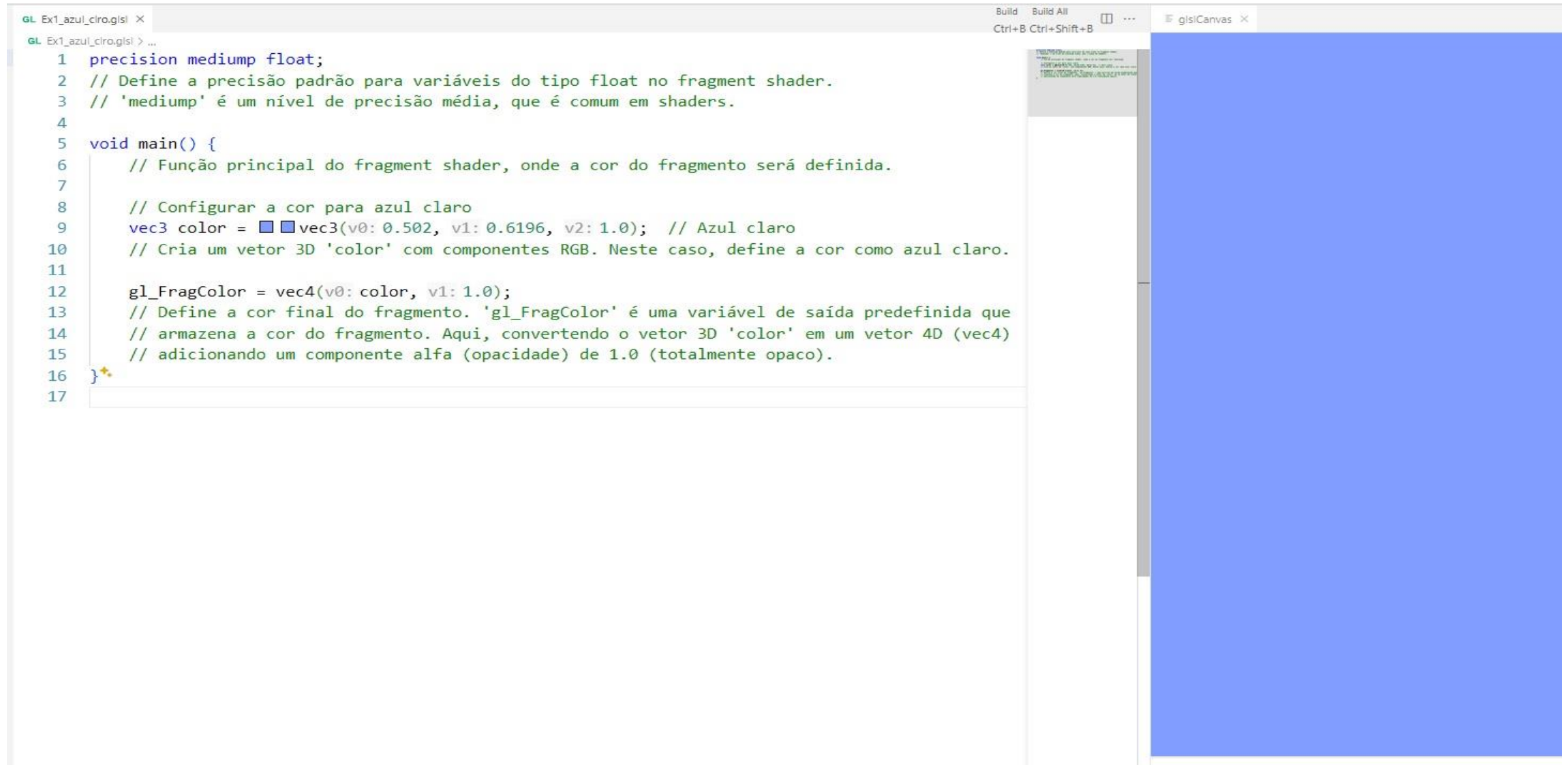
void main() {
    // Função principal do fragment shader, onde a cor do fragmento será definida.

    // Configurar a cor para azul claro
    vec3 color = vec3(0.502, 0.6196, 1.0); // Azul claro
    // Cria um vetor 3D 'color' com componentes RGB. Neste caso, define a cor como azul claro.

    gl_FragColor = vec4(color, 1.0);
    // Define a cor final do fragmento. 'gl_FragColor' é uma variável de saída predefinida que
    // armazena a cor do fragmento. Aqui, convertendo o vetor 3D 'color' em um vetor 4D (vec4)
    // adicionando um componente alfa (opacidade) de 1.0 (totalmente opaco).
}
```

Exemplo 03 – Cor azul

- Resultado



```
GL Ex1_azul_ciro.glsl ×
GL Ex1_azul_ciro.glsl > ...
1 precision mediump float;
2 // Define a precisão padrão para variáveis do tipo float no fragment shader.
3 // 'mediump' é um nível de precisão média, que é comum em shaders.
4
5 void main() {
6     // Função principal do fragment shader, onde a cor do fragmento será definida.
7
8     // Configurar a cor para azul claro
9     vec3 color = vec3(0.502, 0.6196, 1.0); // Azul claro
10    // Cria um vetor 3D 'color' com componentes RGB. Neste caso, define a cor como azul claro.
11
12    gl_FragColor = vec4(color, 1.0);
13    // Define a cor final do fragmento. 'gl_FragColor' é uma variável de saída predefinida que
14    // armazena a cor do fragmento. Aqui, convertendo o vetor 3D 'color' em um vetor 4D (vec4)
15    // adicionando um componente alfa (opacidade) de 1.0 (totalmente opaco).
16 }
17
```

The image shows a code editor window with a GLSL shader file named 'Ex1_azul_ciro.glsl'. The code defines a fragment shader that sets a light blue color. The canvas on the right, titled 'glslCanvas', displays the result of the shader, which is a solid light blue rectangle.

Dados *uniform* disponibilizados

- Cada ferramenta disponibiliza uma série de informações do tipo *uniform* (globais) para desenvolvermos nossos shaders, cada uma com uma nomenclatura diferente.
- Por padrão, as informações fornecidas são:

Tipo	Propriedade	Significado
vec2	u_resolution	Resolução da tela
float	u_time	Tempo, em segundos, desde o início da execução do shader
vec2	u_mouse	Posição do mouse na tela
vec3	u_camera	Posição da câmera
vec4	gl_fragCoord	Coordenadas do fragmento na tela

Dados sobre o objeto disponibilizados

- Além de informações globais, informações sobre objetos também são fornecidas ao shader.
- Em geral, as seguintes informações sobre um objeto serão fornecidas:

Tipo	Propriedade	Significado
vec2	a_position	Posição do objeto
vec4	a_normal	Normal do objeto
vec2	a_texcoord	Coordenada de textura
vec4	a_color	Cor do fragmento

Normalização

- Normalizar é o ato de fazer com que um determinado valor passe a fazer parte de uma nova faixa de valores/dimensão;
- Isso é feito ao aplicarmos uma escala em relação aos limites dessa nova faixa de valores/dimensão.

Normalização Exemplo 04

```
// Define a precisão dos números de ponto flutuante no WebGL.  
precision mediump float;
```

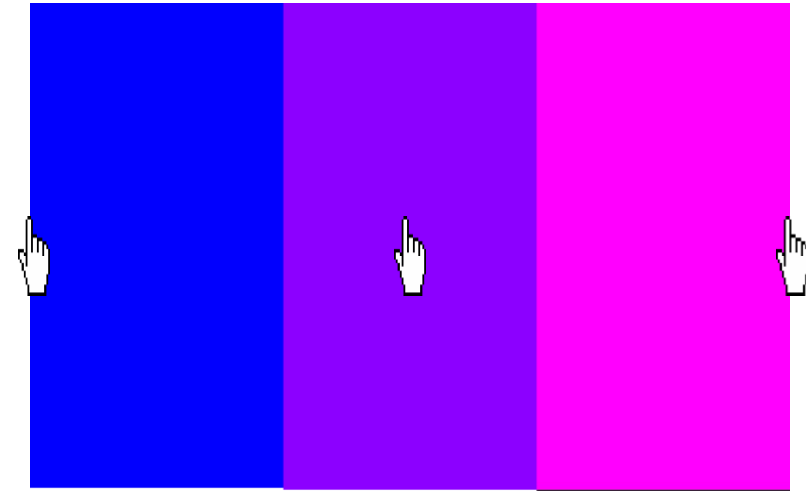
```
// Uniformes que armazenam a resolução da tela e as coordenadas do ponteiro do mouse.  
uniform vec2 u_resolution;  
uniform vec2 u_mouse;
```

```
void main() {  
    // Normalizamos a posição do pixel em relação à resolução da tela,  
    // para que ambas as coordenadas fiquem na mesma faixa de valores (entre 0 e 1).  
    vec2 st = gl_FragCoord.xy / u_resolution;
```

```
    // Normalizamos as coordenadas do ponteiro do mouse em relação à resolução da tela,  
    // para que ambas as coordenadas fiquem na mesma faixa de valores (entre 0 e 1).  
    vec2 mouse_normalized = u_mouse / u_resolution;
```

```
    // Definimos a cor do fragmento usando a coordenada x normalizada do ponteiro do mouse como  
    // o canal vermelho (R),  
    // enquanto os canais verde (G) e azul (B) são fixados em 0. O canal alfa (A) é definido como 1,  
    // tornando o fragmento completamente opaco.  
    gl_FragColor = vec4(mouse_normalized.x, 0.0, 1.0, 1.0);  
}
```

Arrastar o mouse



Produto de Aprendizagem 03 (Peso 3,0)

Testar todos os próximos códigos, arrumar os problemas e deixar funcionando.

Entregar todos os códigos e fazer um breve resumo de cada, por exemplo:

- o código Exemplo 04 muda a cor da tela de azul para rosa ao passar o mouse na direção do eixo x.
- o código Exemplo 06, utiliza a função 'Step' para deixar uma parte da tela na cor preta e a outra na cor branca em relação ao eixo 'x'.

O que deve ser enviado na atividade:

PDF (com as explicações de cada exemplo) + link do github com os códigos.

Exemplo 05 – Ajuste de tamanho?

```
// Define a precisão dos números de ponto flutuante no WebGL.
precision mediump float;

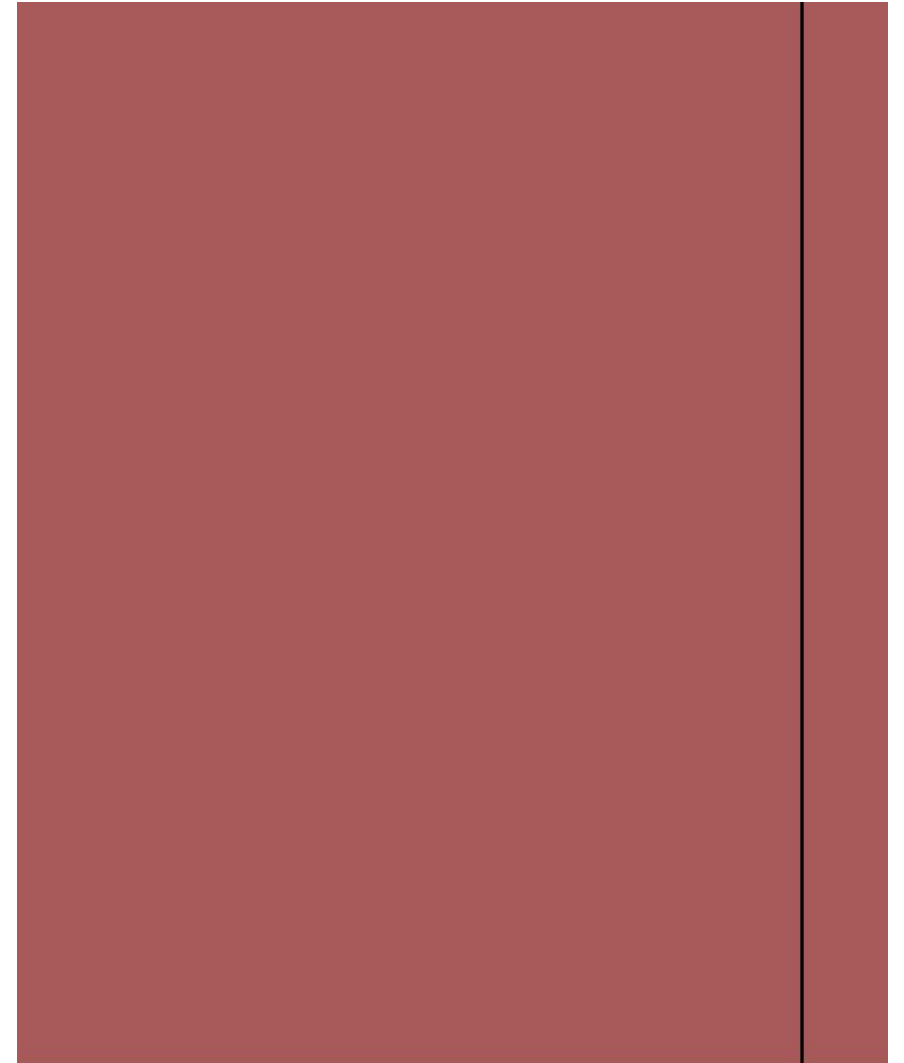
// Uniforme que armazena a resolução da tela.
uniform vec2 u_resolution;

void main() {
    // Normalizamos a posição do pixel em relação à resolução da tela,
    // para que ambas as coordenadas fiquem na mesma faixa de valores
    // (entre 0 e 1).
    vec2 st = gl_FragCoord.xy / u_resolution.xy;

    // Define uma cor inicial (vermelho escuro) para o fragmento.
    vec3 color = vec3(0.6588, 0.349, 0.349);

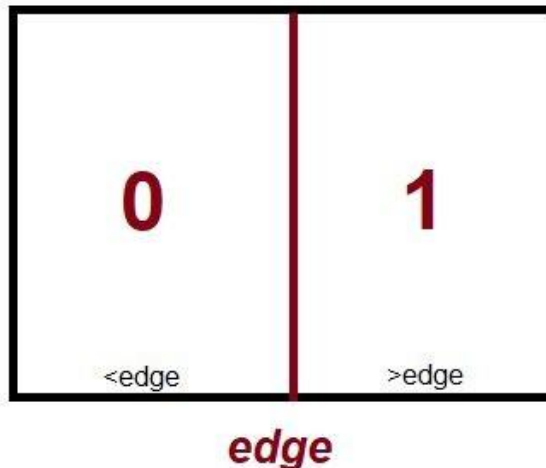
    // Verifica se a coordenada x normalizada do fragmento está dentro
    // de um intervalo específico.
    // Se estiver, define a cor do fragmento como preto (0,0,0), caso
    // contrário, mantém a cor inicial.
    if(st.x > 0.900 && st.x < 0.903){
        color = vec3(0.0); // Define a cor como preto
    }

    // Define a cor final do fragmento com base na cor calculada e
    // define a transparência como 1.0 (completamente opaco).
    gl_FragColor = vec4(color, 1.0);
}
```



Função step

- A GLSL possui funções de interpolação aceleradas por hardware, e estas possuem uma infinidade de aplicações.
- *step* é uma função que recebe 2 parâmetros:
 - O primeiro parâmetro é um limite/limiar (*edge*);
 - O segundo é o valor que desejamos testar (*test*);
 - Se $test < edge$, a função retorna 0.0.
 - Se $test \geq edge$, a função retorna 1.0.



Exemplo 06 – Função *step*

```
// Define a precisão dos números de ponto flutuante no WebGL.
precision mediump float;

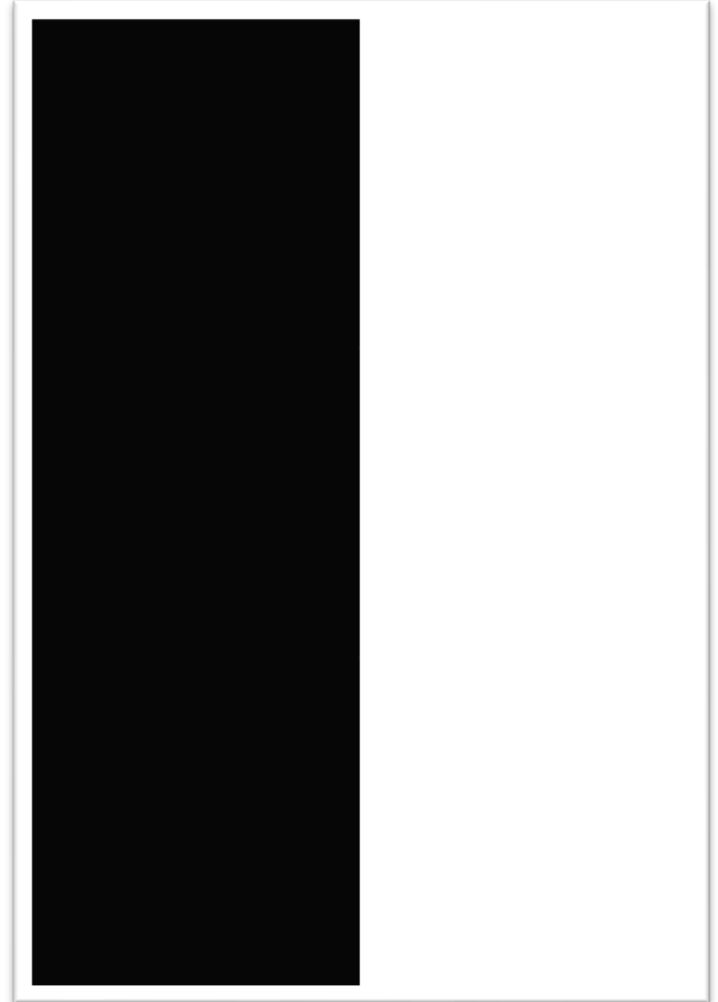
// Uniforme que armazena a resolução da tela.
uniform vec2 u_resolution;

void main() {
    // Normalizamos a posição do pixel em relação à resolução da tela,
    // para que ambas as coordenadas fiquem na mesma faixa de valores (entre 0 e 1).
    vec2 st = gl_FragCoord.xy / u_resolution;

    // Utilizamos a função step para determinar se a coordenada x normalizada (st.x) é maior que 0.5.
    // step(0.5, st.x) retorna 0 caso st.x seja menor que 0.5, e retorna 1 caso seja maior ou igual a 0.5.
    float res = step(0.5, st.x);

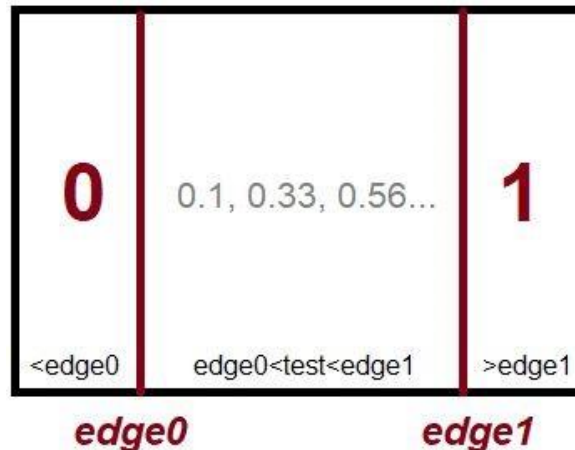
    // Define uma cor baseada no valor calculado pela função step.
    // Se a coordenada x normalizada for maior ou igual a 0.5, a cor será branca (1.0),
    // caso contrário, será preta (0.0).
    vec3 color = vec3(res);

    // Define a cor final do fragmento com base na cor calculada e define a transparência como 1.0
    // (completamente opaco).
    gl_FragColor = vec4(color, 1.0);
}
```



Função smoothstep

- *smoothstep* também é uma função de interpolação, mas que recebe 3 parâmetros:
 - O primeiro parâmetro é o primeiro limite/limiar (*edge0*);
 - O segundo parâmetro é o segundo limite/limiar (*edge1*);
 - O terceiro parâmetro é o valor que desejamos testar (*test*);
 - Se $test < edge0$, a função retorna 0.0.
 - Se $test > edge1$, a função retorna 1.0.
 - Se $edge0 < test < edge1$, a função retorna uma interpolação linear entre 0 e 1



Exemplo 07 - função *smoothstep*

```
// Define a precisão dos números de ponto flutuante no WebGL.
precision mediump float;

// Uniforme que armazena a resolução da tela.
uniform vec2 u_resolution;

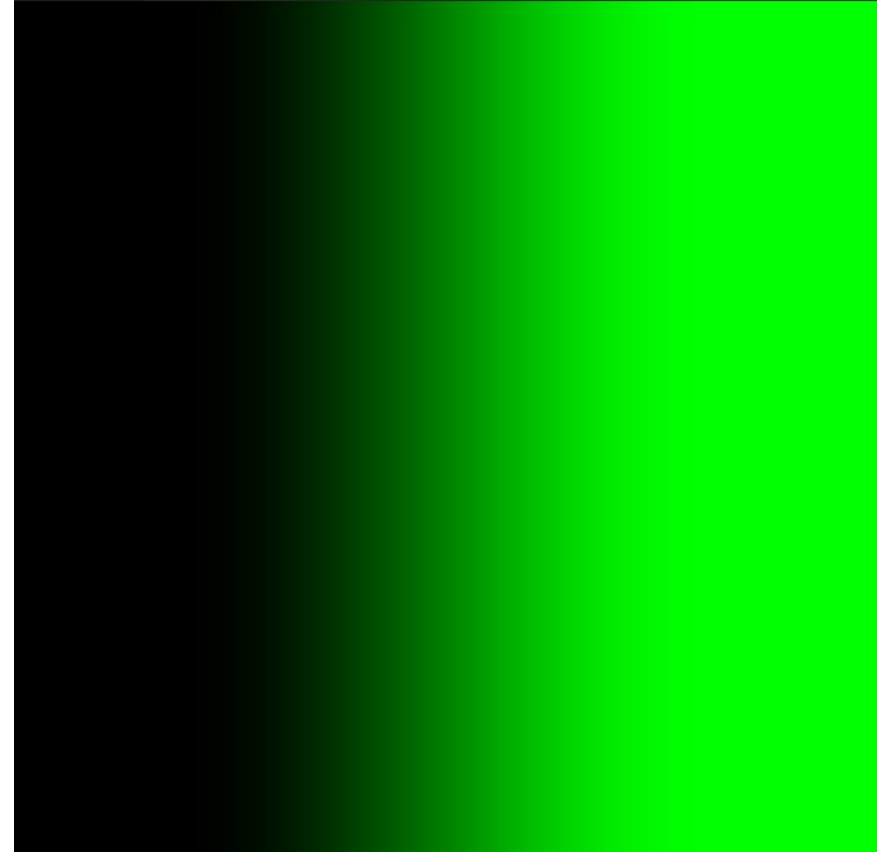
// Função para plotar uma linha suave entre duas posições especificadas.
float plota(vec2 st) {
    // A função smoothstep cria uma transição suave entre dois valores,
    // retornando 0 para valores menores que 0.2, 1 para valores maiores que 0.8,
    // e uma interpolação linear entre 0.2 e 0.8.
    return smoothstep(0.2, 0.9, st.x);
}

void main() {
    // Normalizamos a posição do pixel em relação à resolução da tela,
    // para que ambas as coordenadas fiquem na mesma faixa de valores (entre 0 e 1).
    vec2 st = gl_FragCoord.xy / u_resolution;

    // Calcula a porcentagem de visibilidade (pct) da linha utilizando a função plota.
    float pct = plota(st);

    // Define a cor do fragmento como uma interpolação entre verde (0,1,0) e preto (0,0,0)
    // com base na porcentagem de visibilidade da linha calculada anteriormente.
    vec3 color = pct * vec3(0.0, 1.0, 0.0);

    // Define a cor final do fragmento com base na cor calculada e define a
    // transparência como 1.0 (completamente opaco).
    gl_FragColor = vec4(color, 1.0);
}
```



Exemplo 8 – função *smoothstep*

```
// Define a precisão dos números de ponto flutuante no WebGL.
precision mediump float;

// Uniforme que armazena a resolução da tela.
uniform vec2 u_resolution;

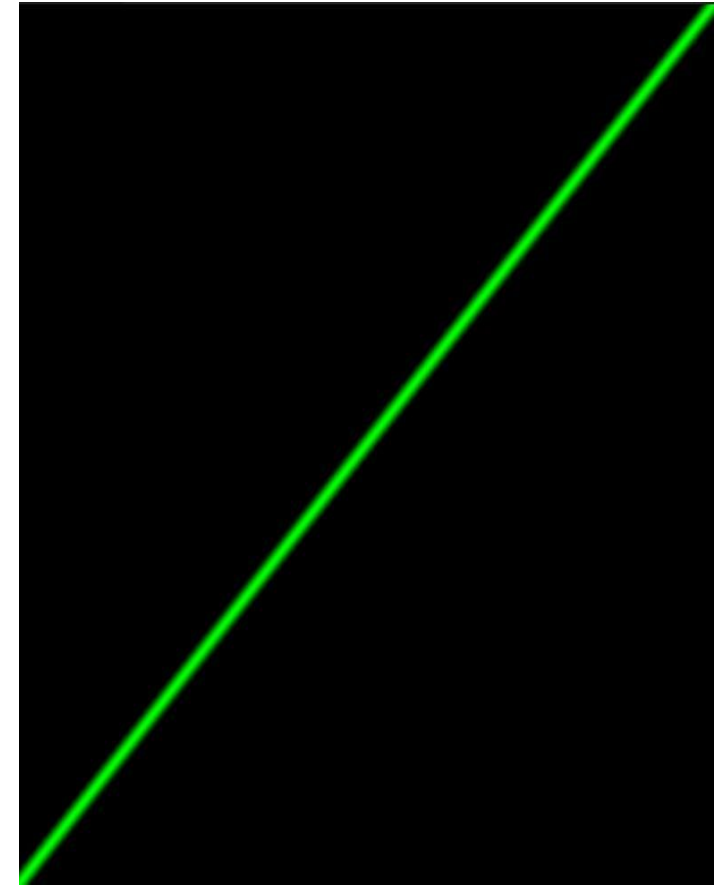
// Função para plotar uma linha suave entre duas posições especificadas.
float plotar(vec2 st, float pct) {
    // A função smoothstep cria uma transição suave entre dois valores.
    // Aqui, v1 faz a transição de 0 para 1 quando st.y está próximo de pct-0.02.
    float v1 = smoothstep(pct - 0.02, pct, st.y);
    // E v2 faz a transição de 0 para 1 quando st.y está próximo de pct+0.02.
    float v2 = smoothstep(pct, pct + 0.02, st.y);
    // Subtraindo v2 de v1, obtemos um valor que é 1 apenas quando st.y está próximo de pct.
    return v1 - v2;
}

void main() {
    // Normalizamos a posição do pixel em relação à resolução da tela,
    // para que ambas as coordenadas fiquem na mesma faixa de valores (entre 0 e 1).
    vec2 st = gl_FragCoord.xy / u_resolution;

    // Desenha uma linha ao longo do eixo x.
    // A variável 'valor' será 1 apenas quando st.y estiver próximo de st.x.
    float valor = plotar(st, st.x);

    // Define a cor do fragmento como verde (0,1,0) quando 'valor' é 1,
    // e preto (0,0,0) quando 'valor' é 0.
    vec3 color = valor * vec3(0.0, 1.0, 0.0);

    // Define a cor final do fragmento com base na cor calculada e define a transparência como 1.0 (completamente opaco).
    gl_FragColor = vec4(color, 1.0);
}
```



Entendendo o exemplo 8

```
#ifdef GL_ES
precision mediump float;
#endif

uniform vec2 u_resolution;

float plota(vec2 st, float pct){
    float v1 = smoothstep( pct-0.02, pct, st.y);
    float v2 = smoothstep( pct, pct+0.02, st.y);
    return v1-v2;
}

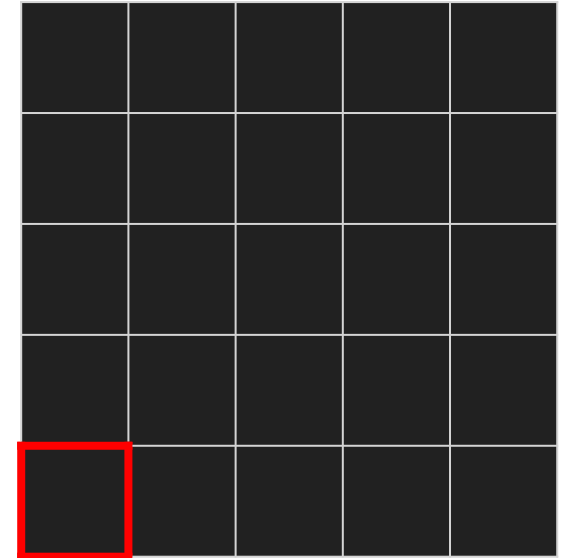
void main() {
    vec2 st = gl_FragCoord.xy/u_resolution;

    // Desenha uma linha
    float valor = plota(st, st.x);
    vec3 color = valor*vec3(0.0,1.0,0.0);

    gl_FragColor = vec4(color,1.0);
}
```

```
st.x = 0.0
st.y = 0.0
pct = 0.0
v1 = smoothstep(-0.02,0.0, 0.0)
v1 = algum valor entre 0 e 1
v2 = smoothstep(0.0,0.02, 0.0)
v2 = algum valor entre 0 e 1

valor = alguma coisa...
```



Entendendo o exemplo 8

```
#ifdef GL_ES
precision mediump float;
#endif

uniform vec2 u_resolution;

float plota(vec2 st, float pct){
    float v1 = smoothstep( pct-0.02, pct, st.y);
    float v2 = smoothstep( pct, pct+0.02, st.y);
    return v1-v2;
}

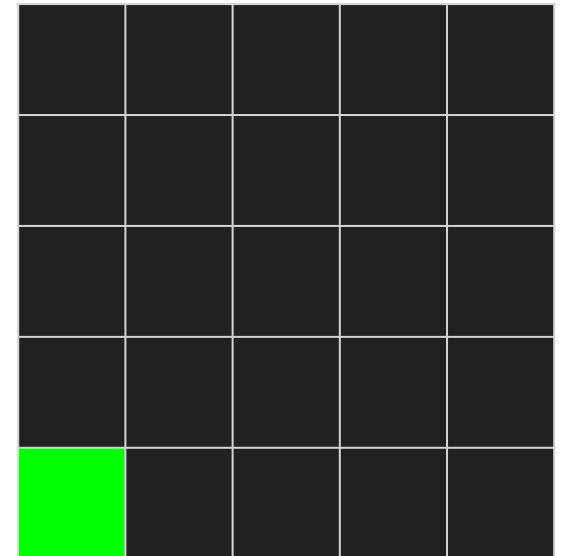
void main() {
    vec2 st = gl_FragCoord.xy/u_resolution;

    // Desenha uma linha
    float valor = plota(st, st.x);
    vec3 color = valor*vec3(0.0,1.0,0.0);

    gl_FragColor = vec4(color,1.0);
}
```

```
st.x = 0.0
st.y = 0.0
pct = 0.0
v1 = smoothstep(-0.02,0.0, 0.0)
v1 = algum valor entre 0 e 1
v2 = smoothstep(0.0,0.02, 0.0)
v2 = algum valor entre 0 e 1

valor = alguma coisa...
```



Entendendo o exemplo 8

```
#ifdef GL_ES
precision mediump float;
#endif

uniform vec2 u_resolution;

float plota(vec2 st, float pct){
    float v1 = smoothstep( pct-0.02, pct, st.y);
    float v2 = smoothstep( pct, pct+0.02, st.y);
    return v1-v2;
}

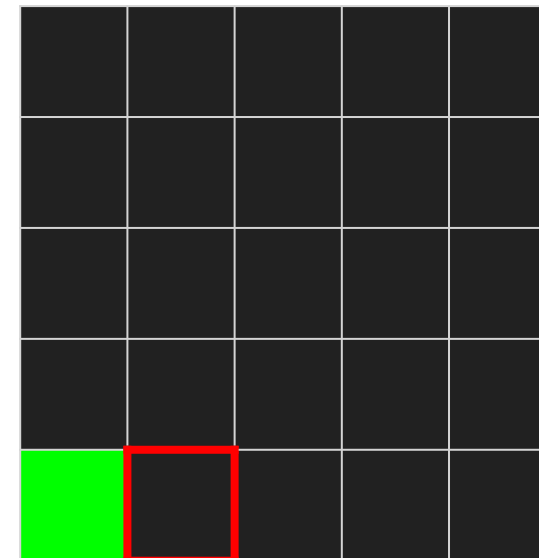
void main() {
    vec2 st = gl_FragCoord.xy/u_resolution;

    // Desenha uma linha
    float valor = plota(st, st.x);
    vec3 color = valor*vec3(0.0,1.0,0.0);

    gl_FragColor = vec4(color,1.0);
}
```

```
st.x = 0.2
st.y = 0.0
pct = 0.2
v1 = smoothstep(0.18,0.2, 0.0)
v1 = 0
v2 = smoothstep(0.2,0.22, 0.0)
v2 = 0

valor = 0 - 0 = 0
```



Entendendo o exemplo 8

```
#ifndef GL_ES
precision mediump float;
#endif

uniform vec2 u_resolution;

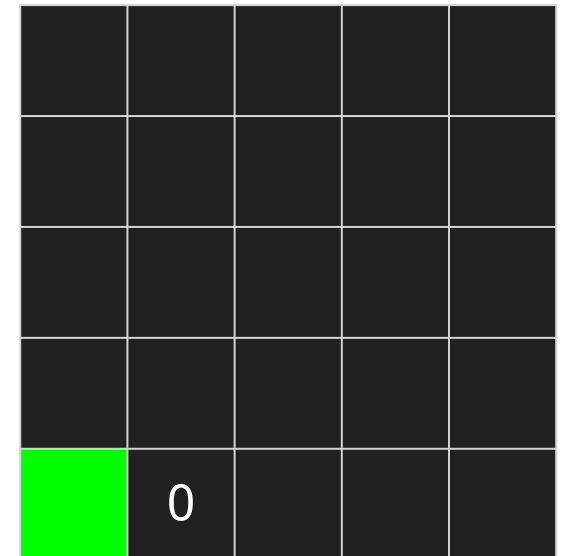
float plota(vec2 st, float pct){
    float v1 = smoothstep( pct-0.02, pct, st.y);
    float v2 = smoothstep( pct, pct+0.02, st.y);
    return v1-v2;
}

void main() {
    vec2 st = gl_FragCoord.xy/u_resolution;

    // Desenha uma linha
    float valor = plota(st, st.x);
    vec3 color = valor*vec3(0.0,1.0,0.0);

    gl_FragColor = vec4(color,1.0);
}
```

```
st.x = 0.2
st.y = 0.0
pct = 0.2
v1 = smoothstep(0.18,0.2, 0.0)
v1 = 0
v2 = smoothstep(0.2,0.22, 0.0)
v2 = 0
valor = 0
```



Entendendo o exemplo 8

```
#ifdef GL_ES
precision mediump float;
#endif

uniform vec2 u_resolution;

float plota(vec2 st, float pct){
    float v1 = smoothstep( pct-0.02, pct, st.y);
    float v2 = smoothstep( pct, pct+0.02, st.y);
    return v1-v2;
}

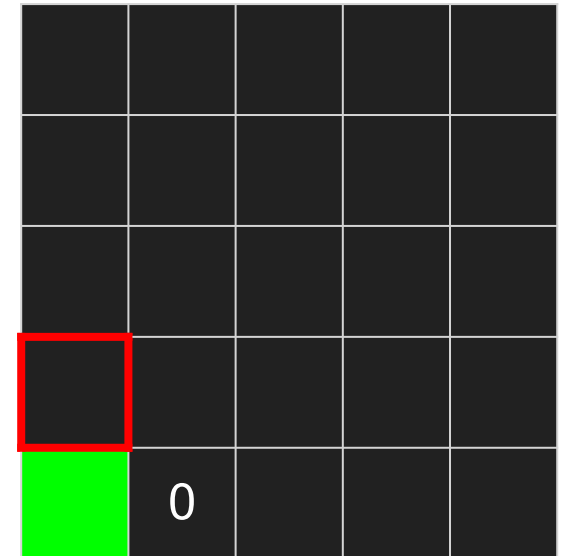
void main() {
    vec2 st = gl_FragCoord.xy/u_resolution;

    // Desenha uma linha
    float valor = plota(st, st.x);
    vec3 color = valor*vec3(0.0,1.0,0.0);

    gl_FragColor = vec4(color,1.0);
}
```

```
st.x = 0.0
st.y = 0.2
pct = 0.0
v1 = smoothstep(-0.02,0.0, 0.2)
v1 = 1
v2 = smoothstep(0.0,0.02, 0.2)
v2 = 1

valor = 1 - 1 = 0
```



Entendendo o exemplo 8

```
#ifdef GL_ES
precision mediump float;
#endif

uniform vec2 u_resolution;

float plota(vec2 st, float pct){
    float v1 = smoothstep( pct-0.02, pct, st.y);
    float v2 = smoothstep( pct, pct+0.02, st.y);
    return v1-v2;
}

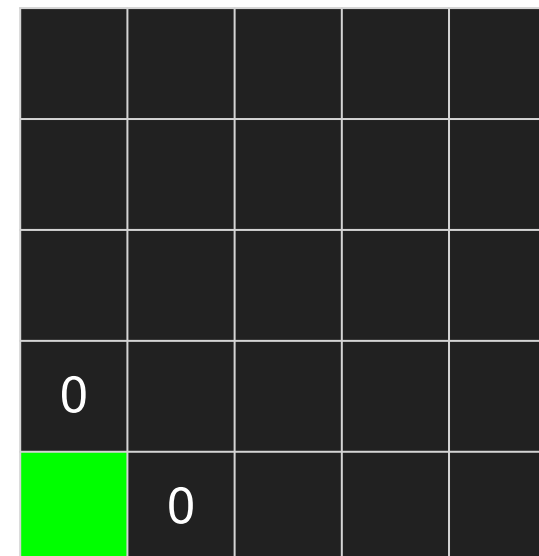
void main() {
    vec2 st = gl_FragCoord.xy/u_resolution;

    // Desenha uma linha
    float valor = plota(st, st.x);
    vec3 color = valor*vec3(0.0,1.0,0.0);

    gl_FragColor = vec4(color,1.0);
}
```

```
st.x = 0.0
st.y = 0.2
pct = 0.0
v1 = smoothstep(-0.02,0.0, 0.0)
v1 = 0
v2 = smoothstep(0.0,0.02, 0.0)
v2 = 0

valor = 0
```



Entendendo o exemplo 8

```
#ifdef GL_ES
precision mediump float;
#endif

uniform vec2 u_resolution;

float plota(vec2 st, float pct){
    float v1 = smoothstep( pct-0.02, pct, st.y);
    float v2 = smoothstep( pct, pct+0.02, st.y);
    return v1-v2;
}

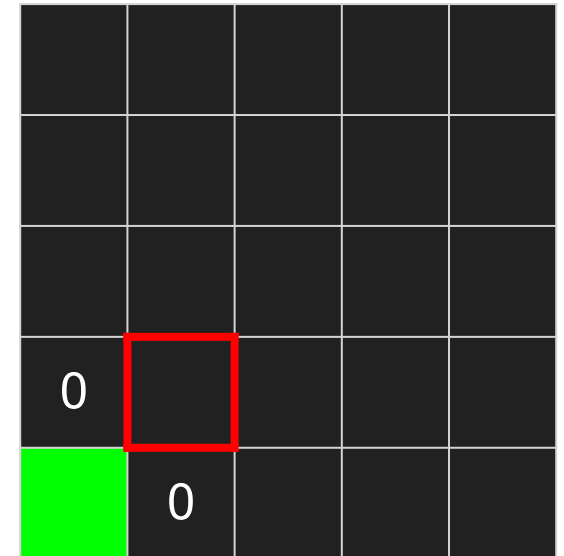
void main() {
    vec2 st = gl_FragCoord.xy/u_resolution;

    // Desenha uma linha
    float valor = plota(st, st.x);
    vec3 color = valor*vec3(0.0,1.0,0.0);

    gl_FragColor = vec4(color,1.0);
}
```

```
st.x = 0.2
st.y = 0.2
pct = 0.2
v1 = smoothstep(0.18,0.2, 0.2)
v1 = algum valor entre 0 e 1
v2 = smoothstep(0.2,0.22, 0.2)
v2 = algum valor entre 0 e 1

valor = alguma coisa...
```



Entendendo o exemplo 8

```
#ifdef GL_ES
precision mediump float;
#endif

uniform vec2 u_resolution;

float plota(vec2 st, float pct){
    float v1 = smoothstep( pct-0.02, pct, st.y);
    float v2 = smoothstep( pct, pct+0.02, st.y);
    return v1-v2;
}

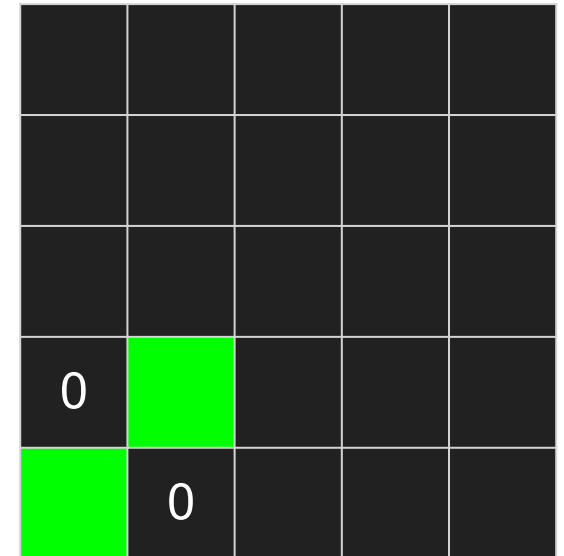
void main() {
    vec2 st = gl_FragCoord.xy/u_resolution;

    // Desenha uma linha
    float valor = plota(st, st.x);
    vec3 color = valor*vec3(0.0,1.0,0.0);

    gl_FragColor = vec4(color,1.0);
}
```

```
st.x = 0.2
st.y = 0.2
pct = 0.2
v1 = smoothstep(0.18,0.2, 0.2)
v1 = algum valor entre 0 e 1
v2 = smoothstep(0.2,0.22, 0.2)
v2 = algum valor entre 0 e 1

valor = alguma coisa...
```



Entendendo o exemplo 8

```
#ifdef GL_ES
precision mediump float;
#endif

uniform vec2 u_resolution;

float plota(vec2 st, float pct){
    float v1 = smoothstep( pct-0.02, pct, st.y);
    float v2 = smoothstep( pct, pct+0.02, st.y);
    return v1-v2;
}

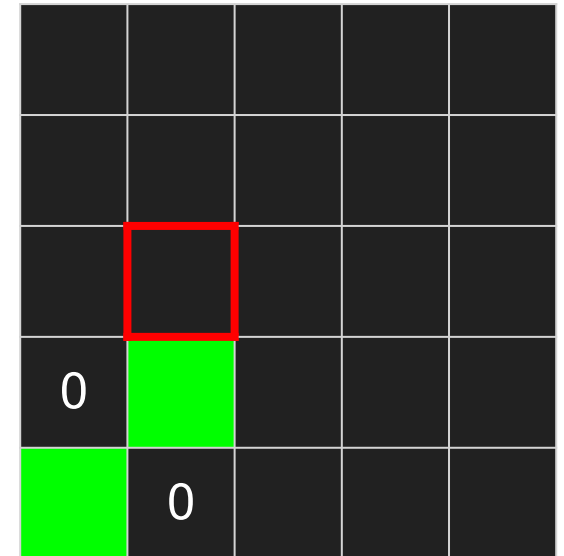
void main() {
    vec2 st = gl_FragCoord.xy/u_resolution;

    // Desenha uma linha
    float valor = plota(st, st.x);
    vec3 color = valor*vec3(0.0,1.0,0.0);

    gl_FragColor = vec4(color,1.0);
}
```

```
st.x = 0.2
st.y = 0.4
pct = 0.2
v1 = smoothstep(0.18,0.2, 0.4)
v1 = 1
v2 = smoothstep(0.2,0.22, 0.4)
v2 = 1

valor = 1 - 1 = 0
```



Entendendo o exemplo 8

```
#ifdef GL_ES
precision mediump float;
#endif

uniform vec2 u_resolution;

float plota(vec2 st, float pct){
    float v1 = smoothstep( pct-0.02, pct, st.y);
    float v2 = smoothstep( pct, pct+0.02, st.y);
    return v1-v2;
}

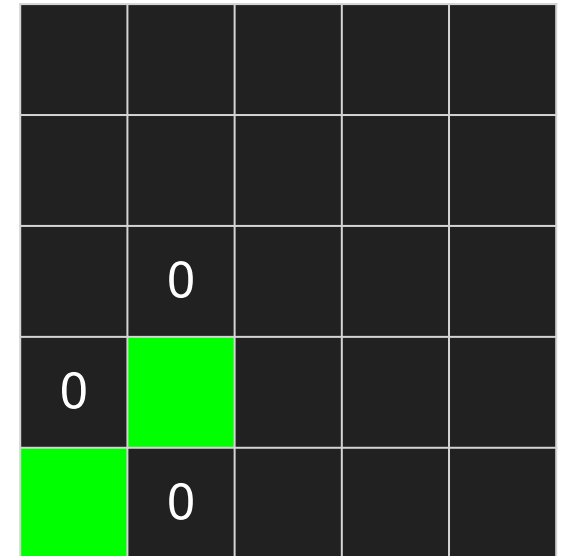
void main() {
    vec2 st = gl_FragCoord.xy/u_resolution;

    // Desenha uma linha
    float valor = plota(st, st.x);
    vec3 color = valor*vec3(0.0,1.0,0.0);

    gl_FragColor = vec4(color,1.0);
}
```

```
st.x = 0.2
st.y = 0.4
pct = 0.2
v1 = smoothstep(0.18,0.2, 0.4)
v1 = 1
v2 = smoothstep(0.2,0.22, 0.4)
v2 = 1

valor = 1 - 1 = 0
```



Entendendo o exemplo 8

e assim por diante...

Exemplo 9 – *shaping functions*

```
// Define a precisão dos números de ponto flutuante no WebGL.
precision mediump float;
// Uniforme que armazena a resolução da tela.
uniform vec2 u_resolution;

// Função para plotar uma linha suave entre duas posições especificadas, usando shaping functions.
float plota(vec2 st, float pct) {
    // A função smoothstep cria uma transição suave entre dois valores.
    // v1 começa a transição de 0 para 1 à medida que st.y se aproxima de pct-0.02.
    float v1 = smoothstep(pct - 0.02, pct, st.y);
    // v2 começa a transição de 0 para 1 à medida que st.y se aproxima de pct+0.02.
    float v2 = smoothstep(pct, pct + 0.02, st.y);
    // Subtraindo v2 de v1, obtemos um valor que é 1 quando st.y está próximo de pct,
    // e uma transição suave de 0 para 1 ao redor dessa linha.
    return v1 - v2;
}

void main() {
    // Normalizamos a posição do pixel em relação à resolução da tela,
    // para que ambas as coordenadas fiquem na mesma faixa de valores (entre 0 e 1).
    vec2 st = gl_FragCoord.xy / u_resolution;

    // Define y como uma função linear da coordenada x (y = x).
    // Isso representa a função f(x) = x, ou seja, uma linha diagonal através da tela.
    float y = st.x;

    // Calcula o valor da linha suave em st utilizando a função plota.
    float valor = plota(st, y);

    // Define a cor da linha como verde (0.0, 1.0, 0.0).
    vec3 lineColor = vec3(0.0, 1.0, 0.0);

    // Define a cor do fundo como branco (1.0, 1.0, 1.0).
    vec3 backgroundColor = vec3(1.0, 1.0, 1.0);

    // A cor final do fragmento é uma interpolação entre a cor da linha e a cor de fundo.
    // Quando retorno for 0 Plota background, quando retorno for 1 plota a linha
    vec3 color = mix(backgroundColor, lineColor, valor);

    // Define a cor final do fragmento com base na cor calculada e define a transparência como 1.0 (completamente opaco).
    gl_FragColor = vec4(color, 1.0);
}
```



Exemplo 9.1 – *shaping functions*

```
precision mediump float;

// Uniforme que armazena a resolução da tela.
uniform vec2 u_resolution;

// Função para plotar uma linha suave entre duas posições especificadas.
float plota(vec2 st, float pct) {
    // A função smoothstep cria uma transição suave entre dois valores.
    // v1 faz a transição de 0 para 1 quando st.y está próximo de pct-0.02.
    float v1 = smoothstep(pct - 0.02, pct, st.y);
    // v2 faz a transição de 1 para 0 quando st.y está próximo de pct+0.02.
    float v2 = smoothstep(pct, pct + 0.02, st.y);
    // Retorna o valor que será 1 no centro da linha e 0 fora dela.
    return v1 - v2;
}

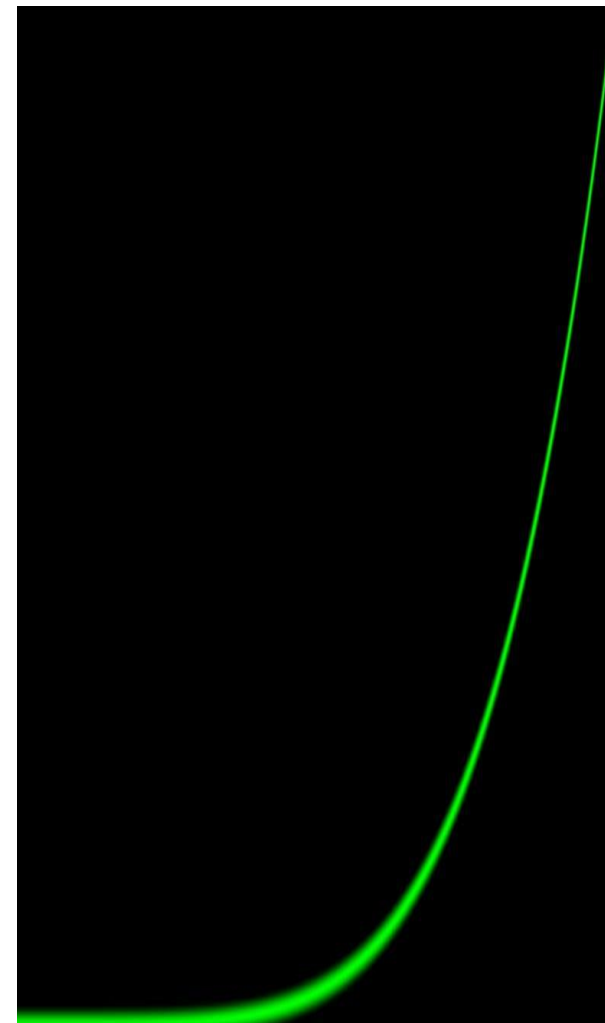
void main() {
    // Normalizamos a posição do pixel em relação à resolução da tela,
    // para que ambas as coordenadas fiquem na mesma faixa de valores (entre 0 e 1).
    vec2 st = (gl_FragCoord.xy / u_resolution);

    // Define y como uma função da coordenada x elevada à quinta potência ( $y = x^5$ ).
    float y = pow(st.x, 2.0);

    // Calcula o valor da linha suave em st utilizando a função plota.
    float valor = plota(st, y);

    // Define a cor da linha como verde (0.0, 1.0, 0.0).
    vec3 color = valor * vec3(0.0, 1.0, 0.0);

    // Define a cor final do fragmento com base na cor calculada e define a transparência como 1.0 (completamente opaco).
    gl_FragColor = vec4(color, 1.0);
}
```



Exemplo 9.2 – *shaping functions*

```
#ifdef GL_ES
precision mediump float;
#endif

// Declaração da resolução da tela
uniform vec2 u_resolution;

// Função para plotar a curva
float plota(vec2 st, float pct){
    // Calcula a primeira parte da curva
    float v1 = smoothstep( pct-0.02, pct, st.y);
    // Calcula a segunda parte da curva
    float v2 = smoothstep( pct, pct+0.02, st.y);
    // Retorna a diferença entre as duas partes, resultando na curva
    return v1 - v2;
}

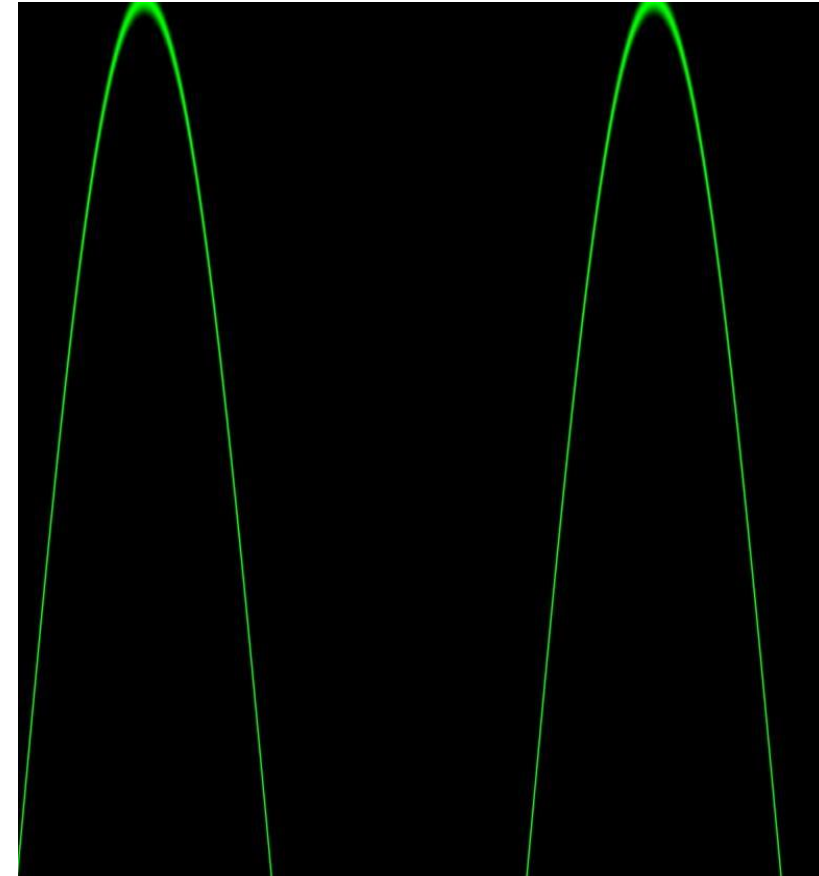
void main() {
    // Normaliza as coordenadas do fragmento para o intervalo [0, 1]
    vec2 st = gl_FragCoord.xy / u_resolution;

    // Calcula o valor da função para o eixo y (seno de 10 vezes x)
    float y = sin(10.0 * st.x);

    // Calcula o valor plotado da função
    float valor = plota(st, y);

    // Define a cor com base no valor plotado da função
    vec3 color = valor * vec3(0.0, 1.0, 0.0);

    // Define a cor final do fragmento
    gl_FragColor = vec4(color, 1.0);
}
```



Exemplo 9.3 – *shaping functions*

```
precision mediump float;

// Declaração da resolução da tela
uniform vec2 u_resolution;

// Função para plotar a curva
float plota(vec2 st, float pct){
    // Calcula a primeira parte da curva
    float v1 = smoothstep(pct - 0.02, pct, st.y);
    // Calcula a segunda parte da curva
    float v2 = smoothstep(pct, pct + 0.02, st.y);
    // Retorna a diferença entre as duas partes, resultando na curva
    return v1 - v2;
}

void main() {
    // Normaliza as coordenadas do fragmento para o intervalo [0, 1]
    vec2 st = gl_FragCoord.xy / u_resolution;

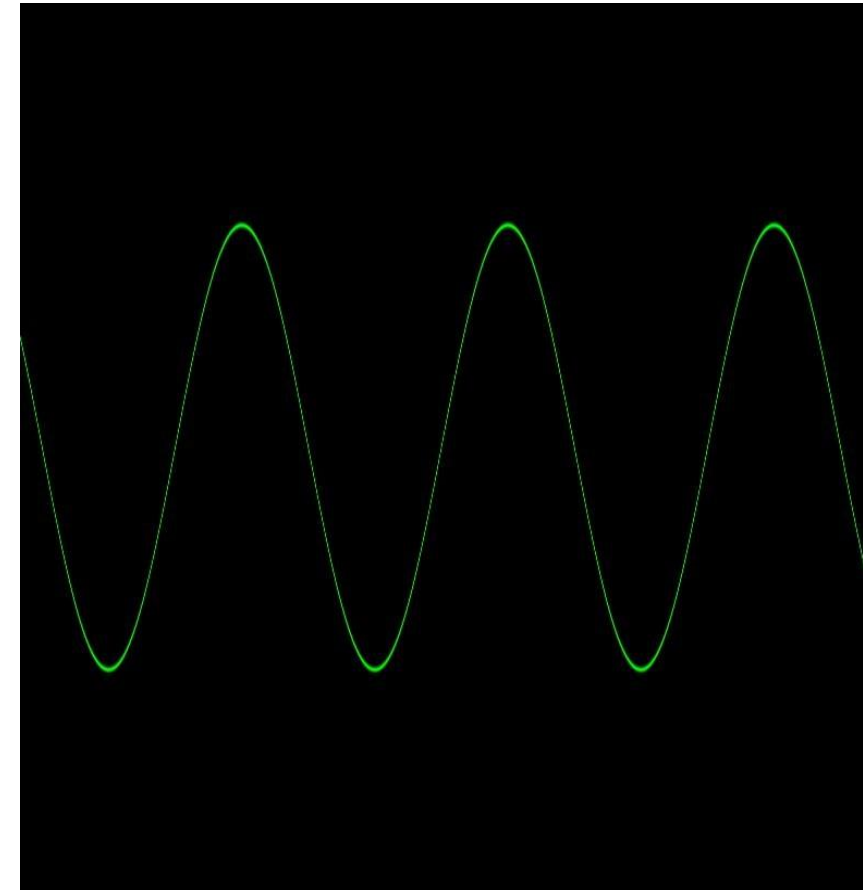
    // Aumenta a área de visualização
    st *= 4.0;
    // Desloca o gráfico da função
    st -= 2.0;

    // Calcula o valor da função para o eixo y (seno de 5 vezes x)
    float y = sin(5.0 * st.x);

    // Desenha uma linha
    float valor = plota(st, y);

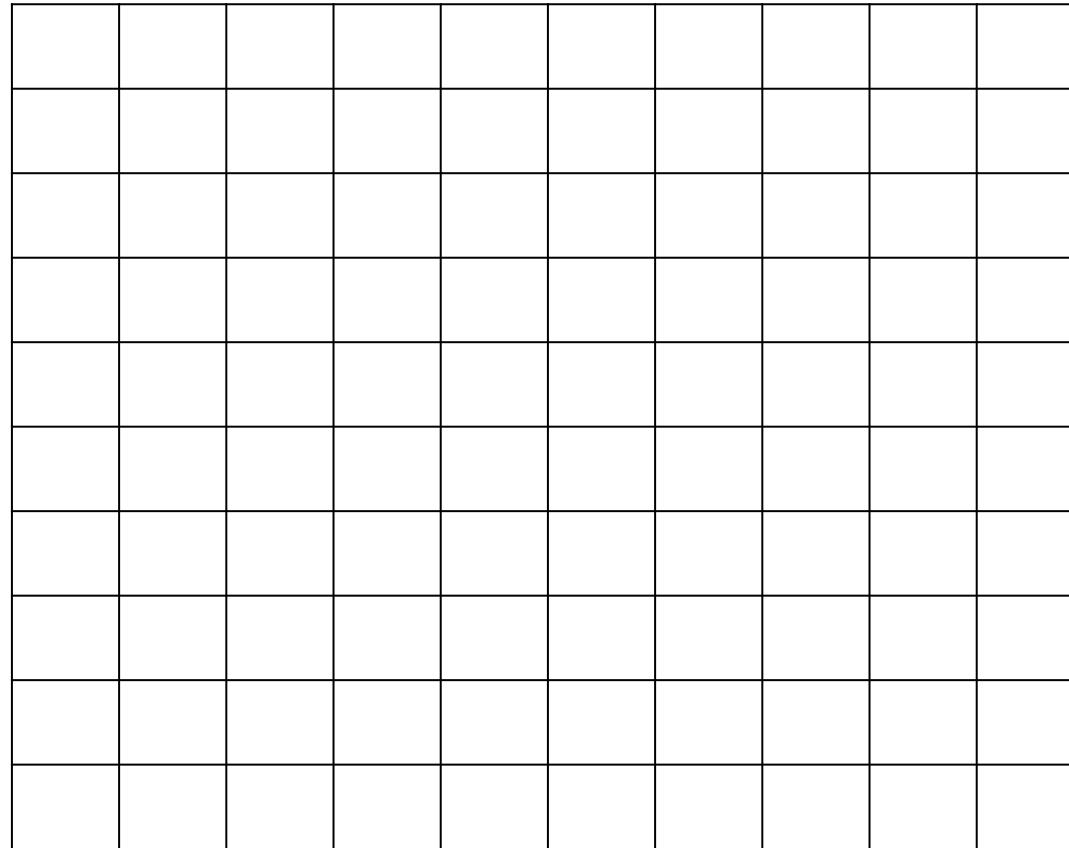
    // Define a cor com base no valor plotado da função
    vec3 color = valor * vec3(0.0, 1.0, 0.0);

    // Define a cor final do fragmento
    gl_FragColor = vec4(color, 1.0);
}
```



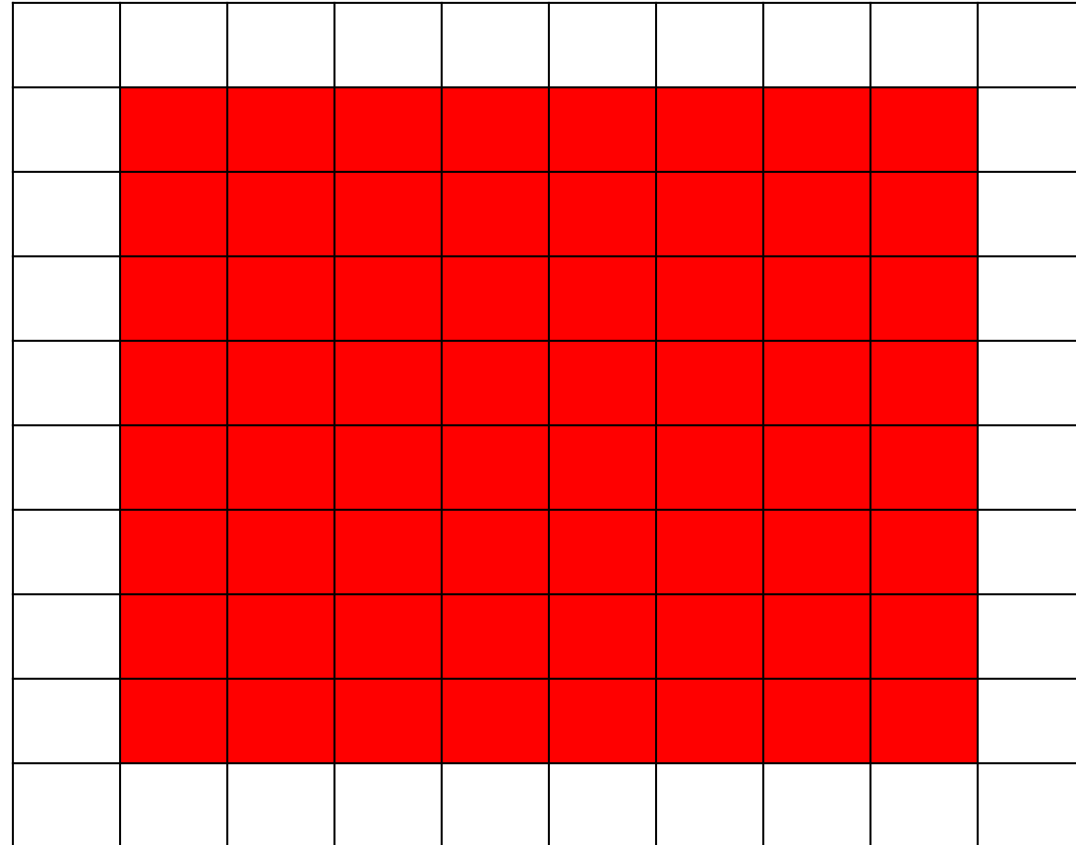
Desenhando formas - Retângulos

- Se você tiver o desafio de desenhar um quadrado de tamanho 8x8 no papel abaixo, cujo tamanho é 10x10, como você faria?



Desenhando formas [2]

- Basta pintarmos tudo, exceto a primeira e última linha, bem como a primeira e última coluna:



Otimizando e implementando, Exemplo10

- Podemos começar a implementar utilizando a função step:

```
#ifdef GL_ES
precision mediump float;
#endif

// Declaração da resolução da tela
uniform vec2 u_resolution;

void main() {
    // Normaliza as coordenadas do fragmento para o intervalo
    [0, 1]
    vec2 st = gl_FragCoord.xy / u_resolution.xy;

    // Cada resultado irá retornar 1.0 (branco) ou 0.0 (preto).
    float left = step(0.1, st.x);    // Se x é maior que 0.1
    float bottom = step(0.1, st.y); // Se y é maior que 0.1

    // A multiplicação left*bottom é similar à operação lógica
    AND.
    vec3 color = vec3(left * bottom);

    // Define a cor final do fragmento
    gl_FragColor = vec4(color, 1.0);
}
```

Finalizando...Exemplo 10_b

```
#ifdef GL_ES
precision mediump float;
#endif

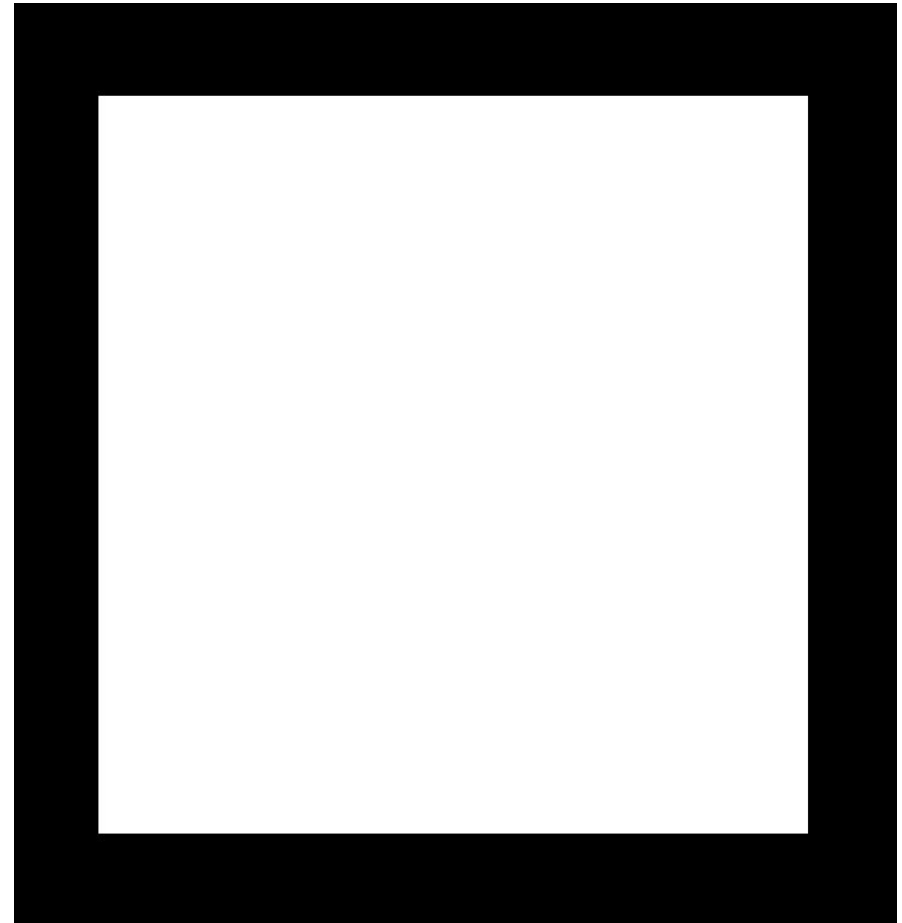
// Declaração da resolução da tela
uniform vec2 u_resolution;
// Posição do mouse
uniform vec2 u_mouse;
// Tempo
uniform float u_time;

void main() {
    // Normaliza as coordenadas do fragmento para o intervalo [0, 1]
    vec2 st = gl_FragCoord.xy / u_resolution.xy;
    // Inicializa a cor como preto
    vec3 color = vec3(0.0);

    // bottom-left
    vec2 bl = step(vec2(0.1), st); // Verifica se o ponto está no
quadrante bottom-left
    // top-right
    vec2 tr = step(vec2(0.1), 1.0 - st); // Verifica se o ponto está
no quadrante top-right

    // Realiza a operação AND
    color = vec3(bl.x * bl.y * tr.x * tr.y);

    // Define a cor final do fragmento
    gl_FragColor = vec4(color, 1.0);
}
```



Círculos

- Desenhar círculos exigem o uso tanto da de equações de distância entre pontos em relação a um ponto de referência, quanto da equação do círculo.

Círculos Exemplo 1

```
#ifdef GL_ES
precision mediump float;
#endif

// Declaração da resolução da tela
uniform vec2 u_resolution;
// Posição do mouse
uniform vec2 u_mouse;
// Tempo
uniform float u_time;

void main() {
    // Normaliza as coordenadas do fragmento para o intervalo [0, 1]
    vec2 st = gl_FragCoord.xy / u_resolution;

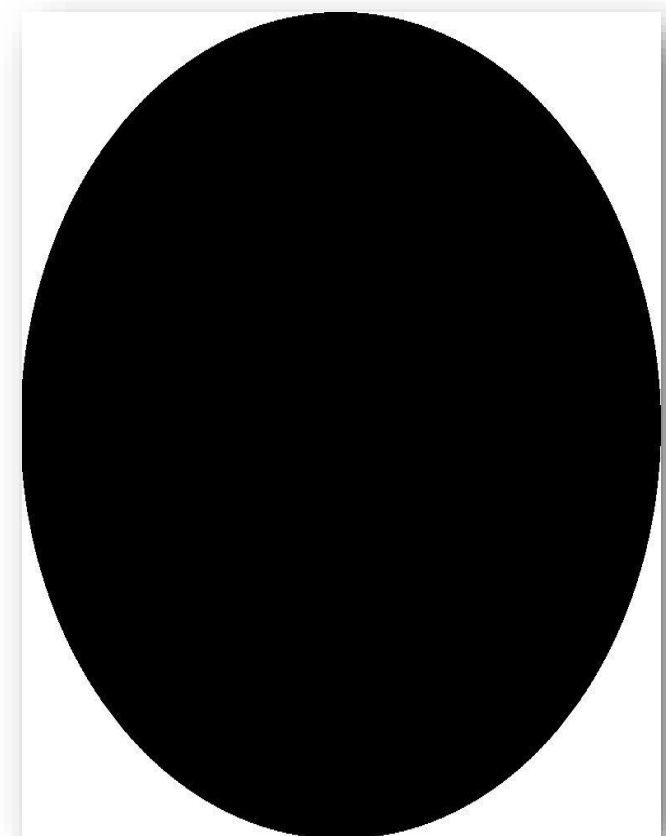
    // Inicializa pct como 0.0
    float pct = 0.0;

    // Calcula a distância do fragmento até o centro do espaço (0.5, 0.5)
    // A função distance() retorna a distância euclidiana entre dois pontos em um plano 2D.
    // No caso, ela retorna a distância entre o ponto st e o ponto (0.5, 0.5), que representa o centro do espaço.
    // A distância euclidiana é calculada usando a fórmula:  $\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$ 
    // Isso significa que a função distance() retorna a magnitude do vetor formado pelos componentes (x, y).
    // Calcula a distância do fragmento até o centro do espaço (0.5, 0.5)
    pct = distance(st, vec2(0.5));

    // Se a distância for maior que 0.5, pct recebe 1.0, caso contrário, 0.0
    pct = step(0.5, pct);

    // Define a cor com base em pct
    vec3 color = vec3(pct);

    // Define a cor final do fragmento
    gl_FragColor = vec4(color, 1.0);
}
```



Criação de padrões – Exemplo 12

```
#ifdef GL_ES
precision mediump float;
#endif

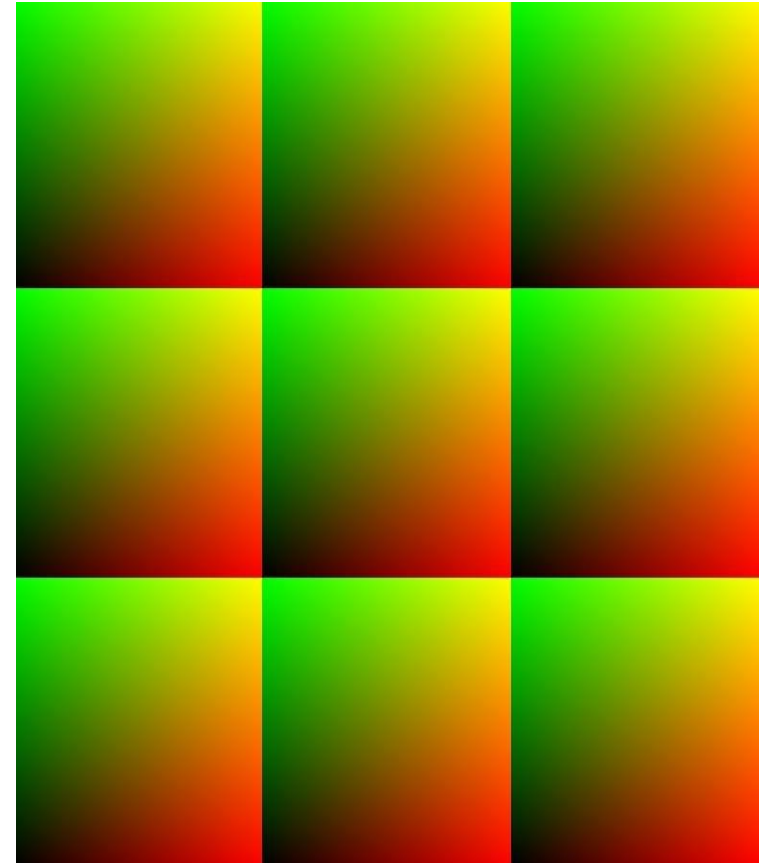
// Declaração da resolução da tela
uniform vec2 u_resolution;
// Tempo
uniform float u_time;

void main() {
    // Normaliza as coordenadas do fragmento para o intervalo [0, 1]
    vec2 st = gl_FragCoord.xy / u_resolution;
    // Inicializa a cor como preto
    vec3 color = vec3(0.0);

    // Aplicamos uma escala de 3x no espaço
    st *= 3.0;
    // Pegamos a parte fracionária das coordenadas
    st = fract(st);

    // Agora temos 9 espaços que vão de 0 a 1, então podemos usar as
    // coordenadas diretamente como cores
    color = vec3(st, 0.0);

    // Define a cor final do fragmento
    gl_FragColor = vec4(color, 1.0);
}
```



Criação de padrões Exemplo13

```
#ifndef GL_ES
precision mediump float;
#endif

// Declaração da resolução da tela
uniform vec2 u_resolution;
// Tempo
uniform float u_time;

// Função para calcular a intensidade de um círculo
float circle(in vec2 _st, in float _radius) {
    // Calcula o vetor de distância do ponto para o centro do círculo
    vec2 l = _st - vec2(0.5);
    // Calcula a distância do ponto para o centro do círculo e suaviza a transição
    return 1.0 - smoothstep(_radius - (_radius * 0.01),
                           _radius + (_radius * 0.01),
                           dot(l, l) * 4.0);
}

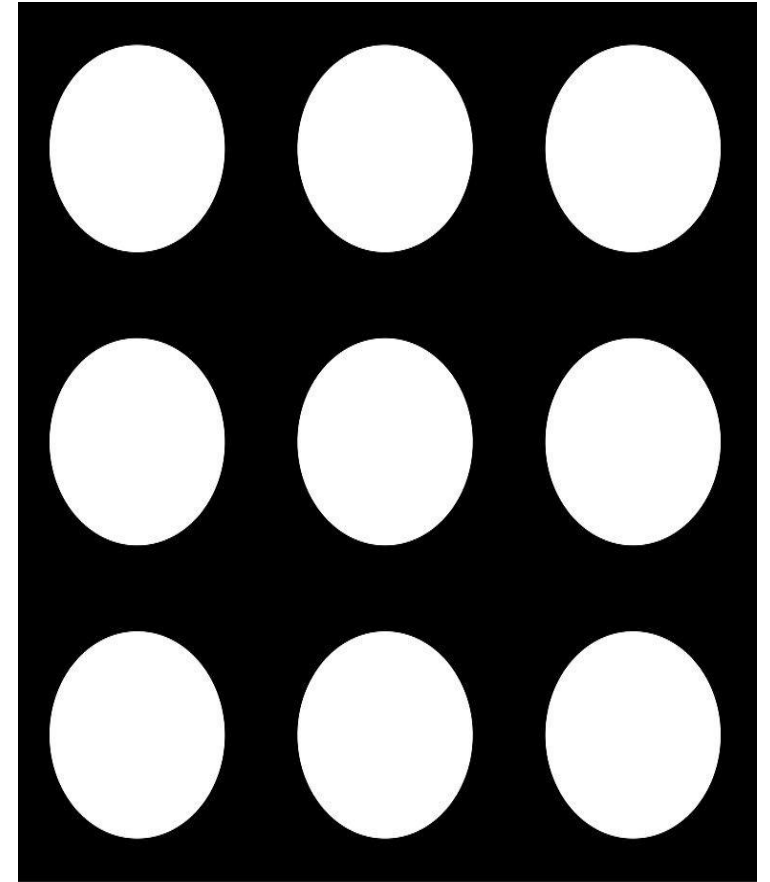
void main() {
    // Normaliza as coordenadas do fragmento para o intervalo [0, 1]
    vec2 st = gl_FragCoord.xy / u_resolution;

    // Inicializa a cor como preto
    vec3 color = vec3(0.0);

    // Aplicamos uma escala de 3x no espaço
    st *= 3.0;
    // Pegamos a parte fracionária das coordenadas
    st = fract(st);

    // Calcula a intensidade do círculo centrado em (0.5, 0.5) com raio 0.5
    color = vec3(circle(st, 0.5));

    // Define a cor final do fragmento
    gl_FragColor = vec4(color, 1.0);
}
```



Criação de padrões Exemplo 14

```
#ifdef GL_ES
precision mediump float;
#endif

// Declaração da resolução da tela
uniform vec2 u_resolution;
// Tempo
uniform float u_time;

// Definição de PI
#define PI 3.14159265358979323846

// Função para rotacionar um ponto 2D em torno da origem
vec2 rotate2D(vec2 st, float angle) {
    // Translada o ponto para a origem
    st -= 0.5;
    // Aplica a rotação usando uma matriz de rotação
    st = mat2(cos(angle), -sin(angle),
              sin(angle), cos(angle)) * st;
    // Translada o ponto de volta
    st += 0.5;
    return st;
}

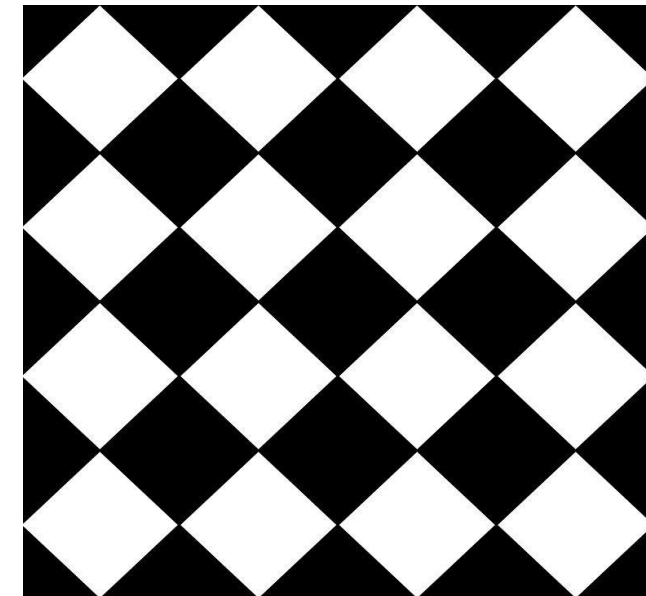
// Função para desenhar um quadrado com bordas suaves
float drawSquare(vec2 st, vec2 size, float smoothEdges) {
    // Calcula o tamanho do quadrado
    vec2 halfSize = vec2(0.5) - size * 0.5;
    // Calcula as bordas suaves
    vec2 edgeSmoothness = vec2(smoothEdges * 0.5);
    // Aplica a suavização nas bordas
    vec2 uv = smoothstep(halfSize, halfSize + edgeSmoothness, st);
    uv *= smoothstep(halfSize, halfSize + edgeSmoothness, vec2(1.0) - st);
    return uv.x * uv.y;
}

// Função para repetir o espaço com um fator de zoom
vec2 tileSpace(vec2 st, float zoomFactor) {
    st *= zoomFactor;
    return fract(st);
}
```

```
void main(void) {
    // Normaliza as coordenadas do fragmento para o intervalo [0, 1]
    vec2 fragCoordNorm = gl_FragCoord.xy / u_resolution.xy;
    // Inicializa a cor como preto
    vec3 color = vec3(0.0);

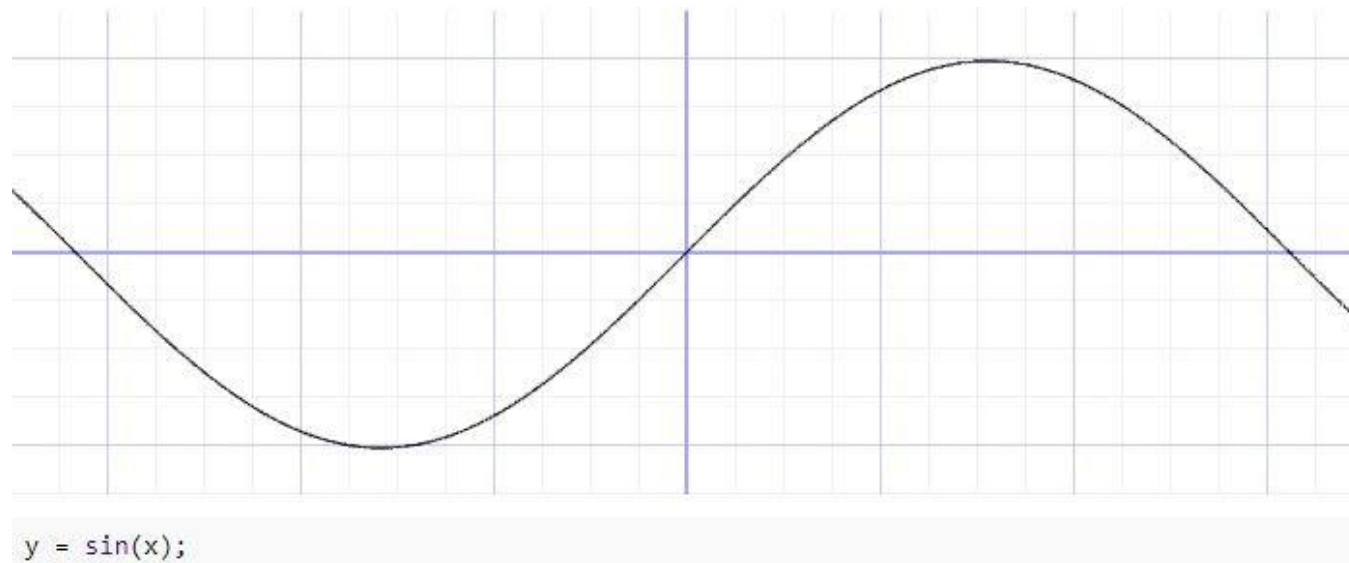
    // Divide o espaço em 4 quadrantes
    vec2 tiledCoords = tileSpace(fragCoordNorm, 4.0);
    // Rotaciona os quadrantes em 45 graus
    vec2 rotatedCoords = rotate2D(tiledCoords, PI * 0.25);
    // Desenha um quadrado com bordas suaves
    color = vec3(drawSquare(rotatedCoords, vec2(0.7), 0.01));

    // Define a cor final do fragmento
    gl_FragColor = vec4(color, 1.0);
}
```



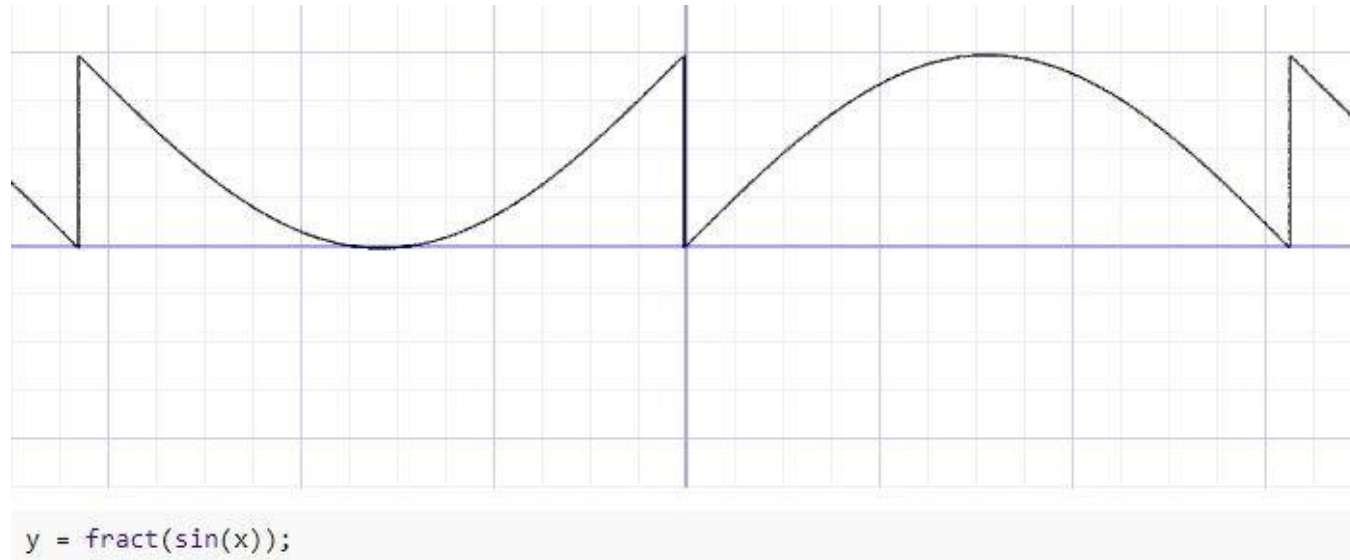
Ruídos e aleatoriedade

- Ao analisarmos a função seno, observamos o seguinte padrão já conhecido:



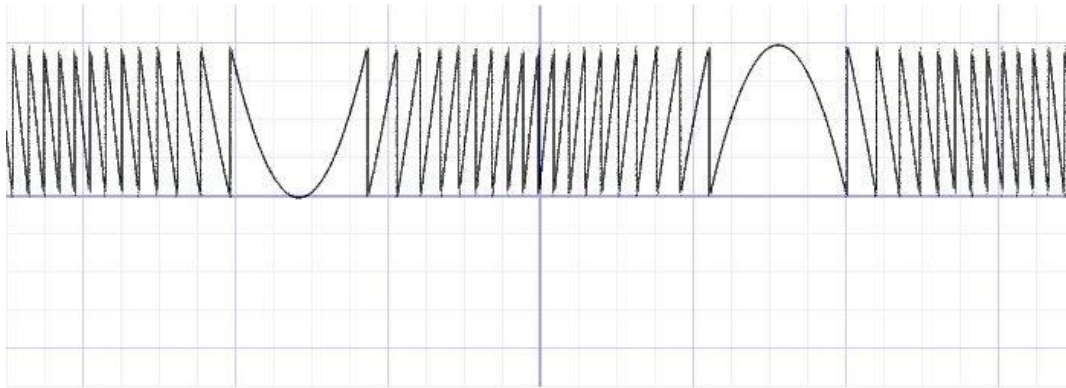
Ruídos e aleatoriedade

- Se pegarmos a parte fracionária como resultado, teremos o seguinte resultado:

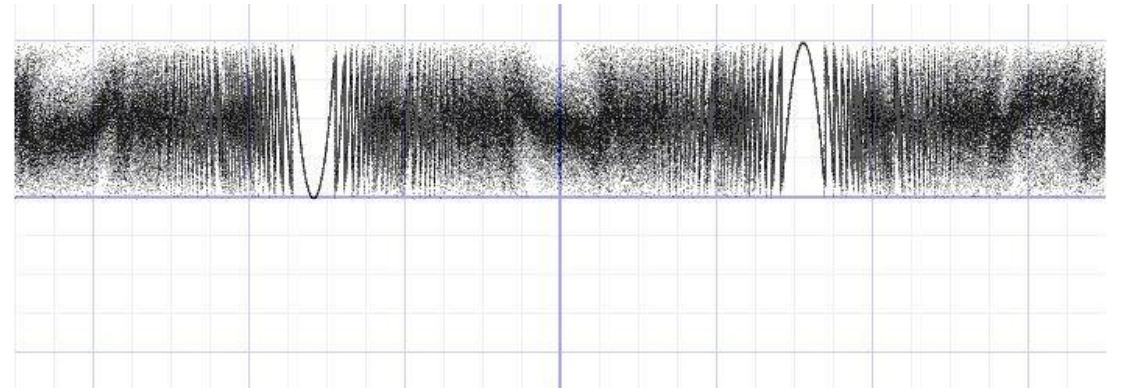


Ruídos e aleatoriedade

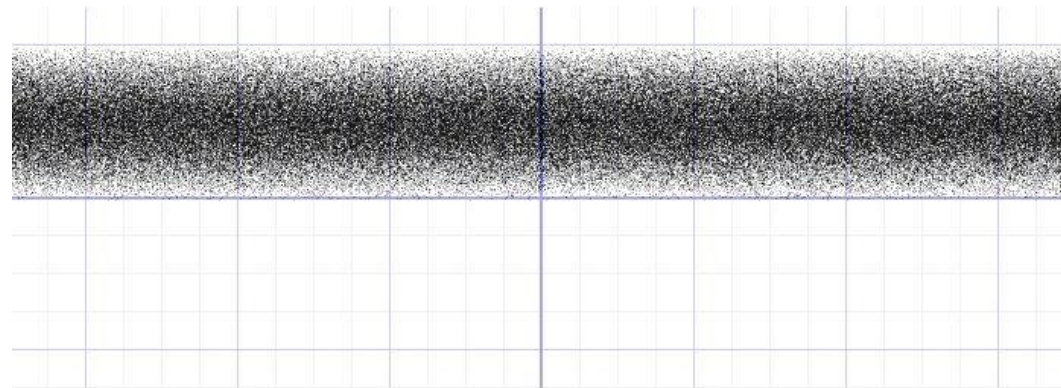
- Se começarmos a multiplicar o seno por números grandes, observe o que acontece...



```
y = fract(sin(x)*10.0);
```



```
y = fract(sin(x)*100.0);
```



```
y = fract(sin(x)*100000.0);
```

E o caos se estabelece...

Ruídos e aleatoriedade Ex15

```
#ifndef GL_ES
precision mediump float;
#endif

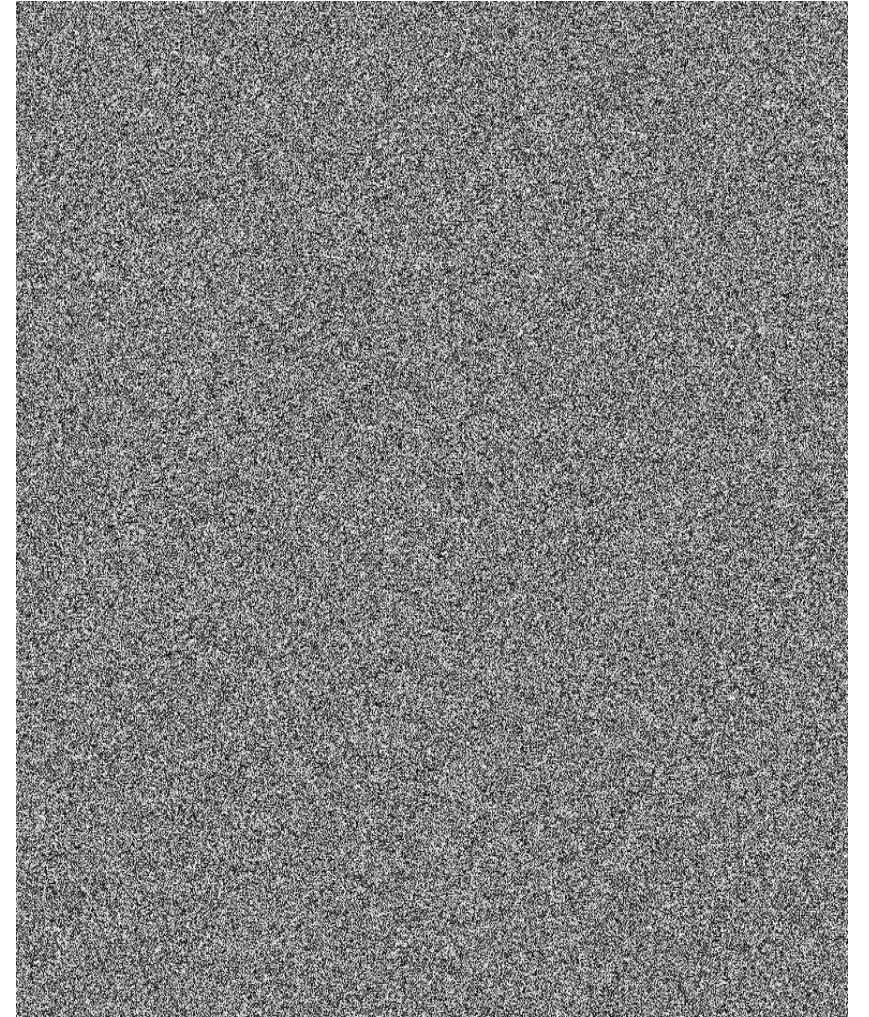
// Declaração da resolução da tela
uniform vec2 u_resolution;
// Posição do mouse
uniform vec2 u_mouse;
// Tempo
uniform float u_time;

// Função para gerar números pseudoaleatórios com base em uma posição
float random(vec2 st) {
    return fract(sin(dot(st.xy, vec2(12.9898, 78.233)) + u_time) *
43758.5453123);
}

void main() {
    // Normaliza as coordenadas do fragmento para o intervalo [0, 1]
    vec2 st = gl_FragCoord.xy / u_resolution.xy;

    // Calcula um valor aleatório com base nas coordenadas do fragmento e no
tempo
    float rnd = random(st) * u_time;

    // Define a cor final do fragmento usando o valor aleatório multiplicado
pelo tempo
    gl_FragColor = vec4(vec3(rnd), 1.0);
}
```



Ruídos e aleatoriedade Exemplo16

```
#ifdef GL_ES
precision mediump float;
#endif

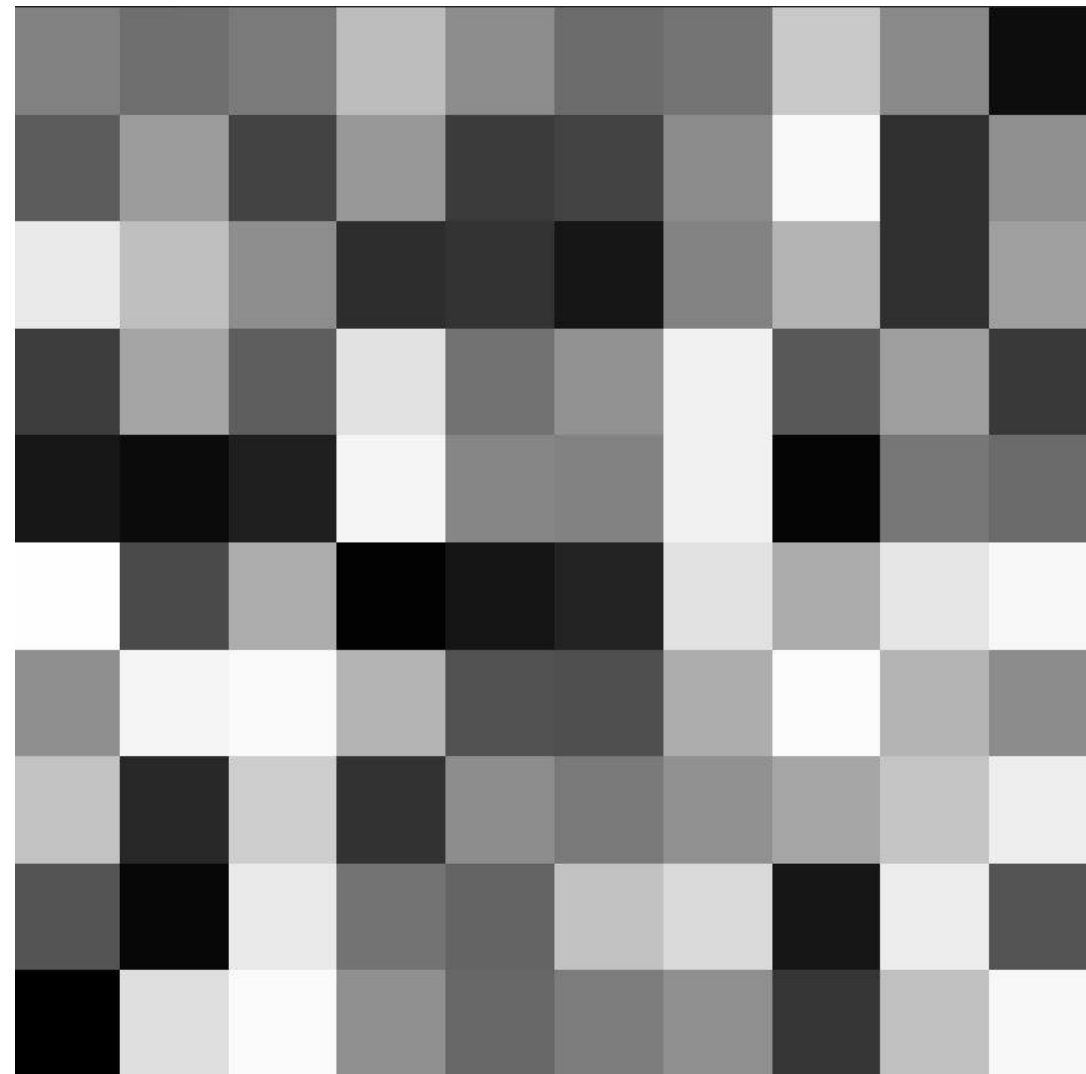
// Declaração da resolução da tela
uniform vec2 u_resolution;
// Posição do mouse
uniform vec2 u_mouse;
// Tempo
uniform float u_time;

// Função para gerar números pseudoaleatórios com base em uma posição
float random(vec2 st) {
    return fract(sin(dot(st.xy, vec2(12.9898, 78.233))) *
43758.5453123);
}

void main() {
    // Normaliza as coordenadas do fragmento para o intervalo [0, 1]
    vec2 st = gl_FragCoord.xy / u_resolution.xy;

    // Aplica uma escala de 10x no sistema de coordenadas
    st *= 10.0;
    // Pega a parte inteira das coordenadas
    vec2 ipos = floor(st);
    // Atribui um valor aleatório com base na coordenada inteira
    vec3 color = vec3(random(ipos));

    // Define a cor final do fragmento
    gl_FragColor = vec4(color, 1.0);
}
```



Ruídos e aleatoriedade [2]

- A função `rand()` da linguagem C é implementada exatamente desta forma;
- Para aplicarmos um ruído aleatório em um espaço bidimensional, podemos utilizar, em conjunto, a função `dot()`, que retorna um float entre 0.0 e 1.0, que varia de acordo com o alinhamento dos vetores:

```
float random (vec2 st) {  
    return fract(sin(dot(st.xy,vec2(12.9898,78.233)))*43758.5453123);  
}
```

Distance fields Exemplo 17

```
#ifdef GL_ES
precision mediump float;
#endif

// Declaração da resolução da tela
uniform vec2 u_resolution;

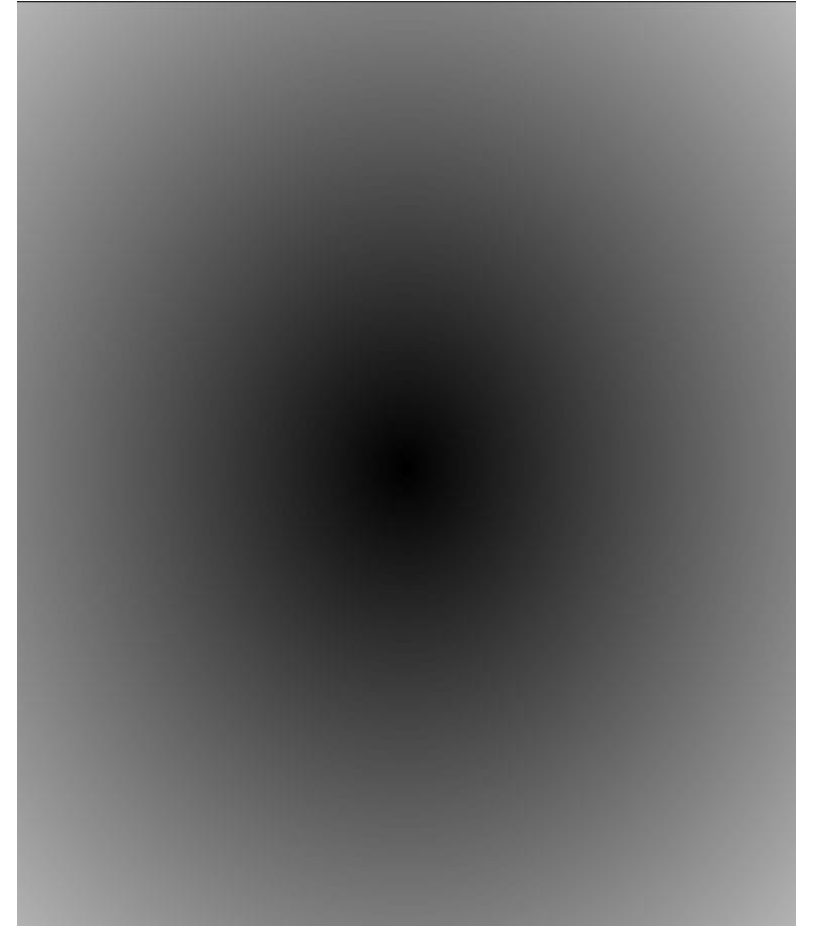
void main() {
    // Normaliza as coordenadas do fragmento para o intervalo [0, 1]
    vec2 st = gl_FragCoord.xy / u_resolution;
    float pct = 0.0;

    // Calcula a distância entre o ponto atual e o ponto (0.5, 0.5)
    pct = distance(st, vec2(0.5));

    /*
    // Calcula o tamanho do vetor do ponto atual até a origem (0,0)
    vec2 centro = vec2(0.5) - st;
    pct = length(centro);
    */

    /*
    // Usando a fórmula euclidiana para calcular a distância entre dois pontos no espaço 2D
    vec2 centro = vec2(0.5) - st;
    pct = sqrt(centro.x * centro.x + centro.y * centro.y);
    */

    // Define a cor final do fragmento com base na distância calculada
    vec3 color = vec3(pct);
    gl_FragColor = vec4(color, 1.0);
}
```



Distance fields Exemplo 18

```
#ifdef GL_ES
precision mediump float;
#endif

uniform vec2 u_resolution;
uniform vec2 u_mouse;
uniform float u_time;

void main(){
    vec2 st = gl_FragCoord.xy/u_resolution;
    float pct = 0.0;

    //Diferentes testes...
    pct = distance(st,vec2(0.4)) + distance(st,vec2(0.6));
    //pct = distance(st,vec2(0.4)) * distance(st,vec2(0.6));
    //pct = min(distance(st,vec2(0.4)),distance(st,vec2(0.6)));
    //pct = max(distance(st,vec2(0.4)),distance(st,vec2(0.6)));
    //pct = pow(distance(st,vec2(0.4)),distance(st,vec2(0.6)));

    vec3 color = vec3(pct);

    gl_FragColor = vec4(color, 1.0 );
}
```

O que acontece quando combinamos diferentes campos de distância, utilizando funções e operações diferentes?

Definição de cores

- Cores podem ser definidas a partir de elementos vetoriais, bem como a partir de seus elementos nominais:

```
vec3 red = vec3(1.0,0.0,0.0); //definição pelo construtor
```

```
//definição elemento por elemento
```

```
red.r = 1.0;
```

```
red.g = 0.0;
```

```
red.b = 0.0;
```


Mistura de cores (Função Mix)

- A **função mix** pode ser utilizada para realizar a mistura de cores:
 - Ela recebe dois vetores e realiza a interpolação linear entre dois valores, de acordo com o terceiro parâmetro.

```
#ifdef GL_ES
precision mediump float;
#endif

uniform float u_time;

vec3 corA = vec3(0.149,0.141,0.912);
vec3 corB = vec3(1.000,0.833,0.224);

void main() {
    vec3 color = vec3(0.0);
    float pct = abs(sin(u_time));

    //color = mix(corA, corB, 0.0); //pega a cor A (0.0)
    //color = mix(corA, corB, 1.0); //pega a cor B (1.0)
    color = mix(corA, corB, pct); //varia entre a (0.0) e B (1.0)
    gl_FragColor = vec4(color,1.0);
}
```

Mistura de cores [2] – Exemplo 19

Se o terceiro parâmetro também for um vetor, ela realiza a interpolação individualmente entre os 3 canais de cores

```
vec3 colorA = vec3(0.149,0.141,0.912);
vec3 colorB = vec3(1.000,0.833,0.224);

float plot (vec2 st, float pct){
    return smoothstep( pct-0.01, pct, st.y) -
           smoothstep( pct, pct+0.01, st.y);
}

void main() {
    vec2 st = gl_FragCoord.xy/u_resolution.xy;
    vec3 color = vec3(0.0);

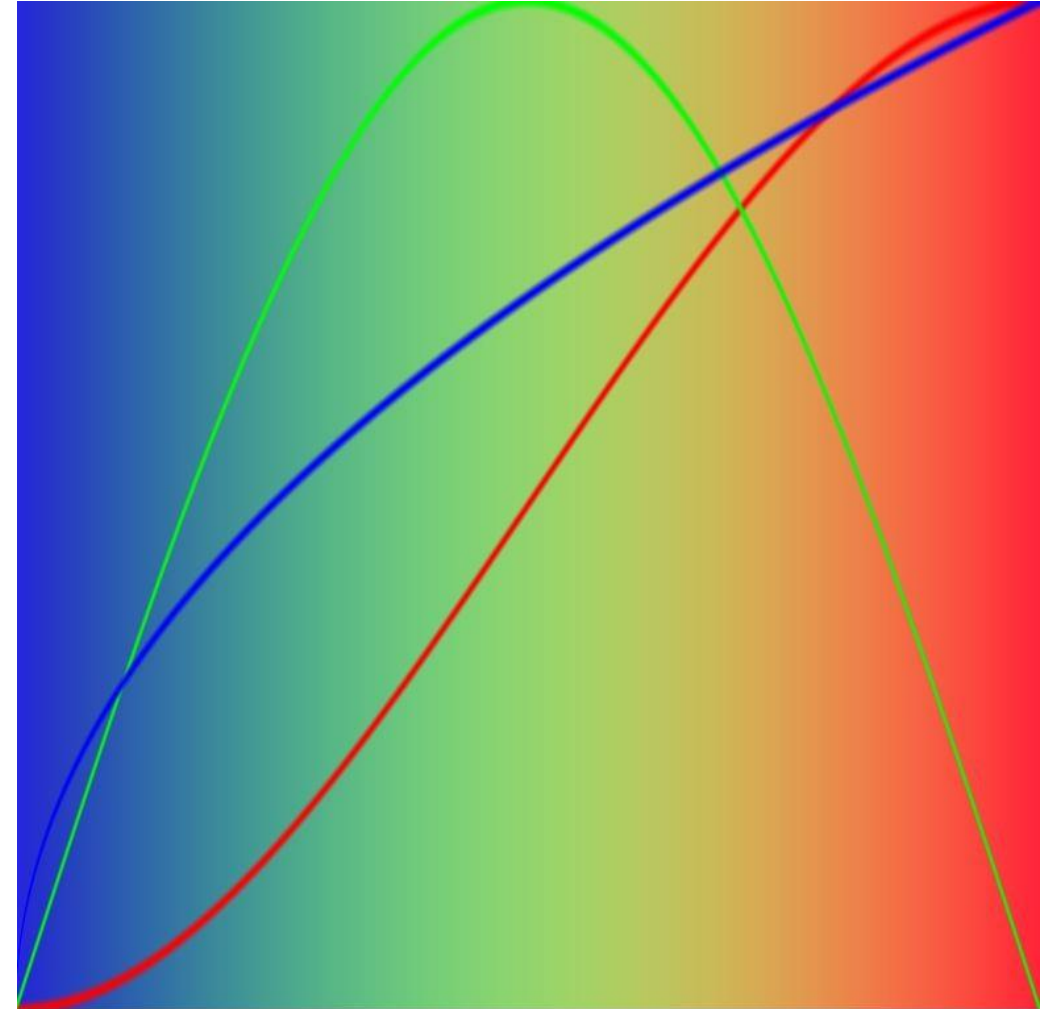
    vec3 pct = vec3(st.x);

    pct.r = smoothstep(0.0,1.0, st.x);
    pct.g = sin(st.x*PI);
    pct.b = pow(st.x,0.5);

    color = mix(colorA, colorB, pct);

    // Plot transition lines for each channel
    color = mix(color,vec3(1.0,0.0,0.0),plot(st,pct.r));
    color = mix(color,vec3(0.0,1.0,0.0),plot(st,pct.g));
    color = mix(color,vec3(0.0,0.0,1.0),plot(st,pct.b));

    gl_FragColor = vec4(color,1.0);
}
```



Referências e material de apoio

- AZEVEDO, Eduardo; CONCI, Aura. Computação gráfica: teoria e prática. Rio de Janeiro: Elsevier, 2003.
- COHEN, Marcelo; MANSSOUR, Isabel H. OpenGL: uma abordagem prática e objetiva. São Paulo: Novatec, 2006.
- FOLEY, J. et al. Computer graphics: principles and practice. Massachusetts: Addison Wesley, 1997
- GOMES, Jonas; VELHO, Luiz. Computação gráfica. Rio de Janeiro: Impa, 1998.
- HEARN, Donald; Baker, M. Pauline. Computer graphics: C version. London: Prentice Hall, 1997.
- HETEM JUNIOR, Annibal. Computação gráfica. Rio de Janeiro, RJ: LTC, 2006. 161 p. (Coleção Fundamentos de Informática).
- HILL Jr, Francis S. Computer graphics using open GL. New Jersey: Prentice Hall, 2001.
- WATT, Alan. 3D computer graphics. Harlow: Addison-Wesley, 2000. Material Prof. Guilherme Chagas Kurtz, 2023.

Thank you for your attention!!



Email: andre.flores@ufn.edu.br